

Symbiosis Institute of Technology, Nagpur

INDUSTRY INTERNSHIP REPORT

Submitted in partial fulfillment of requirement for the award of degree of

Bachelor of Technology
in
Computer Science and Engineering

by

Kritika Nimje
PRN: 22070521193

Industry Guide

Mr. Shrikant Ardhapurkar
Founder

at

Crypto Forensic Technologies

Institute Guide

Dr. Sagarkumar S. Badhiye
HoD, CSE and Associate Professor

May 2026



**Symbiosis International (Deemed University),
Pune**



SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR
Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A++' Grade | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

DECLARATION

I, hereby declare that the Industry Internship report submitted herein has been carried out by me in **(Name of Industry / Organization)** towards partial fulfillment of requirement for the award of Degree of Bachelor of Technology in Computer Science and Engineering. The work is original and has not been submitted earlier as a whole or in part for the award of any degree / diploma at this or any other Institution / University.

I hereby assign all copyright rights in this work, including any revised or expanded derivative works, to Symbiosis Institute of Technology, Nagpur. Any content sourced from references, manuals, or other external materials is duly acknowledged and disclaimed.

Name of Student: **Name1, Name2, Name3**

PRN: **PRN1, PRN2, PRN3**

Signature:

Place: Nagpur

Date: April 17, 2026



SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR
Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A++' Grade | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

CERTIFICATE

This is to certify that the Industry Internship Report, titled “**Case Study(s) / Project Title,**” has been carried out under our supervision at [**Organization/Industry Name**] by [**Student’s Name**] in partial fulfillment of the requirements for the Bachelor of Technology in Computer Science and Engineering. The work submitted is comprehensive, complete and fit for evaluation.

Name

Industry Guide

Name

Institute Guide

Dr. Priya Dasarwar

Internship Incharge

Dr. Sagarkumar S. Badhiye

Head of Department

Dr. Nitin Rakesh

Professor and Director

**(Certificate from industry / organization on industry / organization letter head
describing duration and project work done, project implemented, project sponsored by
industry, achievement & stipend details)**

(On Industry / Organization Letter Head)

Certificate (Sample)

This is to certify that Mr. /Ms. XYZ student of Computer Science and Engineering from Symbiosis Institute of Technology, Nagpur has completed Internship successfully from DD/MM/YYYY to DD/MM/YYYY. During this period, he/she has shown good interest in the assignment/works given to him/her and worked hard. During his/her tenure of internship, he/she was hard working and focused on activities assigned to them.

Students have worked during internship period on following project under the guidance of Mr. ABC, (Designation & Department).

1. A
2. B
3. C

Regards,

Signature

Date:

Name of HR Head

Designation

Industry / Organization Name & Address

NOTE: Reference no/ Document Number is MUST in this certificate

(On Industry / Organization Letter Head)

Certificate

Ref.No./EBOS/Project/2024-25/007 (Example)

Date:

Cost of Industrial Solution Certificate (Mandatory)

This is to certify that Mr. /Ms. ABC students of Computer Science and Engineering from Symbiosis Institute of Technology, Nagpur has completed Internship successfully from DD/MM/YYYY to DD/MM/YYYY. During this period, they have shown good interest in the assignment/works given to them and worked hard.

Students have worked during internship period on following project/industrial problem under the guidance of Mr. ABC,(Designation & Department).

TITLE OF THE PROJECT WORK/DOMAIN/INDUSTRIAL PROBLEM / CASE STUDIES Industry has spent Rs.....*** amount on their ideas/industries problem, which they have successfully implemented.

(*** ENTER NIL AMOUNT IF INDUSTRY DOES NOT AGREE TO MENTION AMOUNT)

Regards,

Signature

Name of HR Head

Designation

Industry / Organization Name & Address, SEAL MUST

NOTE: Reference no/ Document Number is MUST in this certificate

(On Industry / Organization Letter Head)

Certificate

Ref.No./EBOS/Project/2024-25/007 (Example)

Date:

Savings to Industry Certificate (Mandatory)

This is to certify that Mr./Ms. ABC students of Computer Science and Engineering from Symbiosis Institute of Technology, Nagpur has completed Internship successfully from DD/MM/YYYY to DD/MM/YYYY. During this period, they have shown good interest in the assignment/works given to them and worked hard.

Students have worked during internship period on following project/industrial problem under the guidance of Mr. ABC, (Designation & Department).

TITLE OF THE PROJECT WORK/DOMAIN/INDUSTRIAL PROBLEM/ CASE STUDIES Project work submitted by them has the potential to save cost up to Rs Lakh/year. Also, they were entitled for stipend of Rs. *** /- per month along with canteen, transportation & accommodation facilities.

(*** ENTER NIL IN AMOUNT, IF INDUSTRY DOES NOT AGREE TO MENTION AMOUNT)

Regards,

Signature

Name of HR Head

Designation

Industry / Organization Name & Address, SEAL MUST



SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR
Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A++' Grade | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

ACKNOWLEDGEMENT

I express my sincere gratitude to **[Company Name]** for providing me with the opportunity to complete my internship in **[Department/Field/Domain]** and gain valuable industry experience. This internship has been a highly enriching journey that helped me develop both technical and professional skills.

I am deeply grateful to my mentor/supervisor, **[Industry Supervisor's Name]**, for their constant guidance, insightful feedback, and encouragement throughout the internship. Their expertise and support played a crucial role in enhancing my learning experience. I also extend my appreciation to my colleagues and team members at **[Company Name]** for their cooperation, assistance, and for making my internship a pleasant and productive experience.

I am immensely grateful to my academic institution, Symbiosis Institute of Technology, Nagpur, for facilitating this internship opportunity. I extend my heartfelt appreciation to our Director, **Dr. Nitin Rakesh**, for his continuous support and encouragement in providing students with valuable industry exposure. I would also like to express my deepest gratitude to Dr. Sagarkumar S. Badhiye, Head of the Department, Computer Science and Engineering, for his constant support, encouragement, and invaluable guidance throughout my academic journey.

A special note of thanks goes to the **Training and Placement Department** for their efforts in providing internship opportunities and career guidance. I would also like to express my sincere gratitude to **Dr. Priya Dasarwar**, Internship In-Charge, for her support in coordinating the internship process and ensuring a smooth learning experience.

Furthermore, I sincerely appreciate the guidance and motivation provided by my Institute Mentor, **[Institute Mentor's Name]**, whose valuable insights and feedback have greatly contributed to my professional growth during this internship.

Lastly, I would like to acknowledge my family and friends for their unwavering support and encouragement during this period.

Thank you all for your invaluable contributions to my learning and professional growth.

ABSTRACT

Write a 1–2 page summary of the internship project. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

CONTENTS

1	Introduction to Company	1
1.1	About the Company	1
1.2	Historical Background	1
1.3	Location	2
1.4	Operational Structure	2
1.5	Vision and Mission	3
1.6	Products and Software Solutions	3
2	Project Work / Case Study	5
	Project 1: Zomato ETL Pipeline on Databricks	5
	1.1 Introduction	5
	1.2 Problem Identification	5
	1.3 Objectives	6
	1.4 Architecture Overview	7
	1.5 Work Carried Out	8
	1.6 Solution Provided	10
	1.7 Results	11
	1.8 Conclusion	12
	Project 2: RetailPulse Analytics Platform — End-to-End Retail Data Engineer- ing on AWS	13
	2.1 Introduction	13
	2.2 Problem Identification	15
	2.3 Objectives	15
	2.4 Architecture Overview	16
	2.5 Work Carried Out	18
	2.6 Solution Provided	20
	2.7 Results	22
	2.8 Conclusion	22
	Project 3: Pharmaceutical Supply Chain Streaming Analytics Platform	25
	3.1 Introduction	25

3.2 Problem Identification	26
3.3 Objectives	26
3.4 Architecture Overview	27
3.5 Work Carried Out	29
3.6 Solution Provided	32
3.7 Results	33
3.8 Conclusion	35
A Plagiarism Report	39
B AI Similarity Report	41
C Project Screenshots	43

LIST OF FIGURES

2.1	Zomato ETL Pipeline — Medallion Architecture on Databricks	7
2.2	Driver Notebook — Step-wise Execution and Logging	12
2.3	RetailPulse Analytics Platform — End-to-End AWS Architecture	17
2.4	RetailPulse Platform — Spark Processing and BI Output	23
2.5	Pharmaceutical Supply Chain Analytics Platform — Kinesis + Spark Architecture	28
2.6	Detailed diagram of architecture	34
2.7	QuickSight — Cold Chain Compliance and Regional Sales Analysis . . .	35

LIST OF TABLES

1.1	Infocepts Practice Areas and Functions	2
1.2	Infocepts Proprietary Accelerators and Tools	4
2.1	Medallion Architecture Layer Mapping — Zomato ETL Pipeline	8
2.2	Weekly Work Summary — Zomato ETL Project	10
2.3	Pipeline Validation Test Cases — Zomato ETL	13
2.4	Technology Stack — Zomato ETL Pipeline	14
2.5	AWS Services Used — RetailPulse Analytics Platform	18
2.6	Weekly Work Summary — RetailPulse Analytics Platform	20
2.7	Platform Validation Test Cases — RetailPulse Analytics Platform	24
2.8	Data Sources and Schemas — Pharmaceutical Supply Chain Platform	29
2.9	Weekly Work Summary — Pharmaceutical Supply Chain Streaming Platform	32
2.10	Pipeline Validation Test Cases — Pharmaceutical Supply Chain Platform	36

1. INTRODUCTION TO COMPANY

1.1 About the Company

Infocepts is a global, pure-play data and analytics company headquartered in Naperville, Illinois, USA, with delivery centers across India, including Nagpur and Pune. Founded in 2004, Infocepts has established itself as a specialized partner for enterprises seeking to unlock the value of their data through end-to-end data engineering, cloud modernization, business intelligence, and artificial intelligence solutions. The company operates at the intersection of technology and business strategy, enabling organizations across industries to build scalable, reliable, and insight-driven data platforms.

Infocepts serves clients across North America, Europe, and Asia-Pacific, working with Fortune 500 companies and mid-market enterprises alike. Its client portfolio spans industries including retail, food-tech, pharmaceuticals, financial services, healthcare, and manufacturing. The company differentiates itself through deep technical expertise in modern data stack technologies, a strong consulting-first approach, and a commitment to delivering measurable business outcomes rather than isolated technical deliverables.

1.2 Historical Background

Infocepts was established in 2004 with a focused vision: to become a trusted partner for data-driven transformation. In its early years, the company concentrated on business intelligence and reporting services, building competency around tools such as MicroStrategy, Cognos, and Business Objects. As the data landscape evolved through the 2010s, Infocepts strategically pivoted toward modern cloud-native data engineering, investing heavily in platforms such as Amazon Web Services (AWS), Microsoft Azure, Google Cloud Platform (GCP), Snowflake, Databricks, and Apache Spark.

By the mid-2010s, Infocepts had expanded its service offerings to include data lake architecture, real-time streaming pipelines, and machine learning enablement. The company achieved recognition as a certified partner of Databricks, Snowflake, and AWS, reflecting its technical credibility in the market. In 2020, Infocepts launched its proprietary accelerator framework, **DataAscent**, designed to reduce time-to-value for cloud data migrations. Over two decades of growth, the company has grown from a boutique BI shop to a globally

recognized data engineering and analytics firm with over 600 employees worldwide.

1.3 Location

Infocepts operates a globally distributed model with its corporate headquarters in **Naperville, Illinois, USA**. The primary delivery and engineering center is located in **Nagpur, Maharashtra, India**, which serves as the hub for data engineering, cloud platform development, and analytics delivery for international clients. Additional offices are located in **Pune, Maharashtra** and **Hyderabad, Telangana**, supporting consulting, pre-sales, and specialized AI/ML practices. The US offices also include a presence in **New Jersey** for East Coast client engagement.

The Nagpur delivery center, where this internship was conducted, houses cross-functional teams organized around data engineering squads, cloud architecture, BI and visualization, quality assurance, and program management. The center operates across time zones, collaborating in real time with counterparts in the United States and Europe on enterprise-grade data platform projects.

1.4 Operational Structure

Infocepts is organized into competency-based practice areas that operate in close coordination to deliver integrated data solutions. The major functional divisions include:

Table 1.1: Infocepts Practice Areas and Functions

Sr. No.	Practice Area	Core Function
1	Data Engineering	ETL/ELT pipelines, lakehouse architecture, batch and streaming
2	Cloud & Platform	AWS, Azure, GCP infrastructure design and migration
3	Analytics & BI	Dashboards, self-service analytics, KPI frameworks
4	AI & ML	Predictive modeling, NLP, MLOps pipelines
5	Data Governance	Data quality, cataloging, lineage, and compliance
6	Consulting	Strategy, roadmap definition, architecture review

Each practice area is led by a Principal Architect or Practice Head, supported by senior engineers, data engineers, analysts, and associate engineers. Projects are executed in Agile sprints, typically two weeks in duration, with daily standups, sprint reviews, and retrospectives conducted using tools such as Jira, Confluence, and Microsoft Teams. Interns

are embedded directly within engineering squads, contributing to active client deliverables under the supervision of a senior engineer and an industry mentor.

1.5 Vision and Mission

Vision: To be the most trusted global partner for enterprises on their data-driven transformation journey, enabling every organization to derive strategic value from their data assets.

Mission: To deliver scalable, intelligent, and cloud-native data solutions that empower organizations to make faster, smarter, and more confident decisions. Infocepts is committed to combining deep technical expertise with a consulting mindset, ensuring that every data platform built is aligned with measurable business outcomes and long-term operational sustainability.

These guiding principles are reflected in the company's internal culture, which emphasizes continuous learning, client centricity, engineering excellence, and collaborative innovation. Internally, Infocepts maintains an open knowledge-sharing environment through its **Data Engineering Guild**, regular tech talks, certification drives (AWS, Databricks, Snowflake), and structured mentorship programs for early-career engineers.

1.6 Products and Software Solutions

Infocepts does not operate as a product company in the traditional sense; rather, it offers a portfolio of proprietary accelerators, frameworks, and solution templates that complement its services. Key intellectual property assets include:

Beyond these accelerators, Infocepts delivers custom-built solutions leveraging best-in-class platforms including **Databricks Lakehouse**, **AWS Glue**, **Amazon Redshift**, **Snowflake**, **Apache Spark Structured Streaming**, **Amazon Kinesis Data Streams**, **Delta Lake**, and **dbt (data build tool)**. The company also maintains strategic alliances with Tableau, MicroStrategy, and Alteryx for analytics and visualization delivery.

During the internship period, the intern was actively involved in three client-facing data engineering projects spanning food-tech, retail, and pharmaceutical domains, each leveraging a distinct subset of these technologies and accelerators. The following chapters document the technical work carried out, the architecture designed, and the outcomes achieved across these engagements.

Table 1.2: Infocepts Proprietary Accelerators and Tools

Sr. No.	Accelerator / Product	Description
1	DataAscent	End-to-end cloud data migration framework supporting lift-and-shift and re-architecture patterns for AWS, Azure, and GCP
2	PipelineIQ	Modular ETL/ELT template library built on Databricks and Apache Spark, enabling rapid pipeline scaffolding
3	InsightHub	Self-service analytics and BI enablement layer integrating with Tableau, Power BI, and Looker
4	DataQualityX	Automated data profiling and quality validation framework with rule engine and alerting capabilities
5	StreamEdge	Real-time streaming pipeline accelerator built on Apache Kafka and Amazon Kinesis

2. PROJECT WORK / CASE STUDY

Project 1: Zomato ETL Pipeline on Databricks

1.1 Introduction

The food-tech industry generates enormous volumes of semi-structured data daily, encompassing restaurant metadata, user ratings, cuisine classifications, geographic attributes, and delivery capabilities. For platforms such as Zomato, this data serves as the backbone of business intelligence, enabling country-wise performance benchmarking, cuisine trend analysis, pricing strategy, and operational decision-making. However, raw data in its native JSON format is analytically unusable without a reliable, governed, and scalable transformation pipeline.

This project, undertaken during the internship at Infocepts, involved the end-to-end design and implementation of a production-grade ETL (Extract, Transform, Load) pipeline on **Databricks Free Edition**, following modern data engineering best practices. The pipeline ingests semi-structured JSON data from Zomato restaurant metadata, transforms it through multiple processing layers, and loads it into **Delta Lake** tables managed under **Unity Catalog**. The final output consists of analytics-ready, partitioned summary tables suitable for downstream reporting and business consumption.

The solution was architected around the **Medallion Architecture** (Bronze → Silver → Gold), a widely adopted data lakehouse design pattern that enforces separation of concerns and data quality at each processing stage. The pipeline is orchestrated through a centralized **Driver Notebook** that controls execution flow, logs audit records, enforces concurrency protection via a lock mechanism, and supports multiple load strategies — scheduled, manual, and historical.

1.2 Problem Identification

Zomato restaurant data is sourced as nested, multi-level JSON files containing arrays of restaurant objects. Each object encapsulates attributes spanning location, cuisine, pricing, rating, and delivery metadata. In its raw form, this data presents several critical engineering and governance challenges that prevent direct analytical consumption:

(i) **Semi-structured Format:** Incoming JSON files are deeply nested, with restaurant

attributes embedded within multi-level object hierarchies (e.g., `restaurant.location.city`, `restaurant.user.rating.aggregate_rating`). Without explicit flattening and schema enforcement, downstream consumers cannot reliably query this data.

(ii) Absence of Data Quality Controls: Raw records frequently contain NULL cuisine values, blank string fields, and binary flag columns encoded as integers. Loading such data into a reporting layer directly introduces analytical inaccuracies, distorts aggregations, and undermines trust in the dataset.

(iii) Duplicate Records: Repeated pipeline executions or source re-extractions can introduce duplicate restaurant records for the same filedate, leading to inflated row counts and incorrect business metrics.

(iv) Lack of Governance and Lineage: Without centralized metadata management, tables created outside a governed namespace are difficult to discover, audit, or access-control. There is also no systematic mechanism to track which files have been processed, when, and by whom.

(v) No Incremental Load Support: A naive full-overwrite strategy on each pipeline run is computationally expensive and operationally fragile. An enterprise pipeline must support safe, idempotent incremental loads at the partition level.

(vi) No Operational Safety: Without concurrency controls, two simultaneous pipeline executions could corrupt Delta tables through conflicting writes, and without audit logging, failures become difficult to diagnose.

These challenges collectively defined the scope of the engineering problem addressed by this project.

1.3 Objectives

The primary objectives established for this project were as follows:

- Implement a multi-stage, modular ETL pipeline entirely within Databricks, following the Medallion Architecture (Bronze → Silver → Gold).
- Leverage **Unity Catalog** for centralized governance, schema management, and table discovery across all pipeline assets.
- Apply **Delta Lake** as the storage layer to ensure ACID-compliant writes, scalable storage, and time-travel capability.
- Enforce comprehensive **data quality rules** — including NULL filtering, deduplication, string standardization, and schema enforcement — at the transformation stage.

- Derive business-critical computed columns, specifically `m_rating_colour` (rating band classification) and `m_cuisines` (cuisine category), to enrich the analytical dataset.
- Support **incremental, manual, and historical load strategies** to accommodate operational flexibility, backfills, and recovery scenarios.
- Implement **audit logging** using a dedicated log table (`zomato_summary_log`) and a **lock mechanism** to ensure concurrency safety and operational observability.
- Design the pipeline for **daily scheduled execution**, producing country-wise and date-wise partitioned summary tables ready for BI consumption.

1.4 Architecture Overview

The pipeline is built on the **Databricks Lakehouse** platform and follows the Medallion Architecture, structured across three Delta Lake layers. The complete workflow is governed by Unity Catalog and orchestrated through a central Driver Notebook. Figure 2.1 illustrates the high-level architecture.

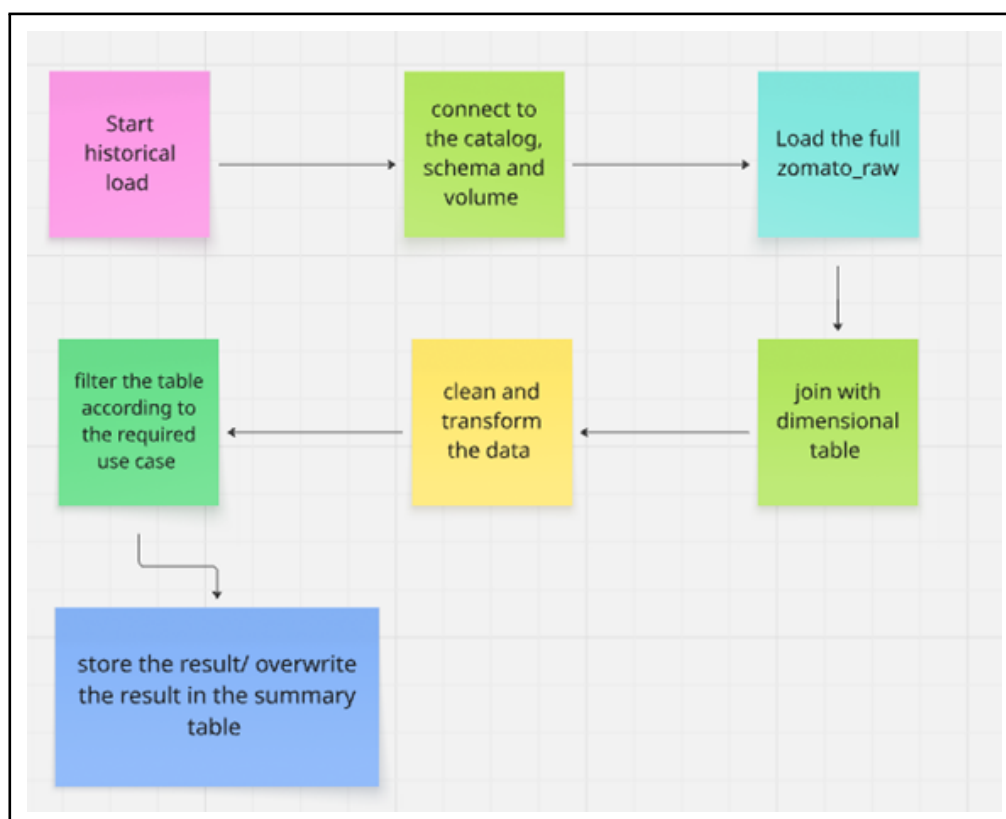


Figure 2.1: Zomato ETL Pipeline — Medallion Architecture on Databricks

The architecture is structured across four logical tiers:

Tier 1 — Ingestion Layer (Databricks Volumes): Raw JSON files land in the `zomato_raw_files` folder within a Unity Catalog-managed Volume (`zomato_files`). A controlled folder structure separates raw inputs, staged JSON files, generated CSVs, archives, and execution logs.

Tier 2 — Bronze Layer (`zomato_raw`): Structured CSV data (converted from JSON in Module 1) is loaded into the `zomato.etl.zomato_raw` Delta table. This layer retains data in its minimally transformed state, partitioned by `filedate`, and serves as the immutable source of truth for all downstream processing.

Tier 3 — Dimension Layer (`dim_country`): A country reference table is maintained as a Delta Lake dimension, populated from a static reference CSV. This table enriches raw records with human-readable country names and enables partition derivation by country in the Gold layer.

Tier 4 — Gold Layer (`zomato_summary`): The final analytics-ready table is produced by applying all business rules — filtering, deduplication, null handling, computed column derivation, and audit column addition — and is stored as a Delta table partitioned by `p_filedate` and `p_country_name`.

All table creation, schema management, and data access are governed through the `zomato` catalog and `etl` schema under Unity Catalog, ensuring enterprise-grade data governance throughout the pipeline.

Table 2.1: Medallion Architecture Layer Mapping — Zomato ETL Pipeline

Layer	Table / Asset	Format	Purpose
Bronze	<code>zomato.etl.zomato_raw</code>	Delta Lake	Raw structured ingestion, partitioned by <code>filedate</code>
Dimension	<code>zomato.etl.dim_country</code>	Delta Lake	Country reference lookup for enrichment
Gold	<code>zomato.etl.zomato_summary</code>	Delta Lake	Analytics-ready, business-rule-applied output
Audit	<code>zomato.etl.zomato_summary_log</code>	Delta Lake	ETL execution metadata and step-level logging

1.5 Work Carried Out

The internship contribution to this project spanned the complete lifecycle of the ETL pipeline, from environment setup through module development, orchestration design, and validation. The work was organized across five structured phases.

Phase 1 — Environment Setup (Environment Setup Notebook): The pipeline environment was initialized by creating the Unity Catalog hierarchy (`zomato catalog, etl schema`), provisioning the managed Volume (`zomato_files`), and establishing the folder structure for raw inputs, staged files, CSVs, archives, and logs. All four Delta tables (`zomato_raw, dim_country, zomato_summary, zomato_summary_log`) were defined with explicit schemas, partitioning strategies, and Delta Lake storage properties. This notebook was designed as an idempotent, run-once setup that can also be re-executed safely at the start of each pipeline run.

Phase 2 — Module 1: JSON to CSV Conversion: This module handles the first stage of data movement. Incoming JSON files are identified dynamically in the raw landing zone, copied to the controlled processing directory (`source/json`), and parsed using PySpark's multiline JSON reader. The nested restaurant array is exploded and flattened into a tabular structure containing exactly 20 mandatory business-approved fields. A `filedate` column is derived by extracting a date pattern from the filename or defaulting to the current system date. The resulting DataFrame is written as a single named CSV file (`zomato-<filedate>.csv`) using `coalesce(1)`, and processed JSON files are archived to `source/archive` to prevent reprocessing.

Phase 3 — Module 2: Raw Table and Dimension Table Loading: CSV files produced by Module 1 are read with schema enforcement derived directly from the `zomato_raw` table definition. An incremental load strategy is implemented: distinct `filedate` values in the incoming data are identified, existing Delta partitions for those dates are deleted via targeted `DELETE FROM` statements, and fresh data is appended. This pattern ensures idempotent, safe daily reruns with no duplicate accumulation. The country dimension table (`dim_country`) is refreshed via a full overwrite of the reference CSV, standardizing column names to match the warehouse schema.

Phase 4 — Module 3: Summary Table Transformation and Loading: This is the core transformation module. The raw table and dimension table are joined on `country_code` using a `LEFT JOIN` to preserve all raw records even where country enrichment is unavailable. Seven mandatory business rules are then applied sequentially: (i) filtering records with `NULL` or blank `cuisines`; (ii) replacing `NULL` values with "NA" across all string columns; (iii) deduplicating records on the composite key (`restaurant_id, filedate`); (iv) deriving `m_rating_colour` based on five rating bands; (v) deriving `m_cuisines` based on cuisine keyword classification; (vi) appending audit columns (`create_datetime, user_id`); and (vii) renaming columns to partition keys (`p_filedate, p_country_name`). The enriched DataFrame is appended into `zomato_summary`.

Phase 5 — Orchestration via Driver Notebook: A central Driver Notebook was implemented to orchestrate all pipeline modules in sequence. It supports runtime widgets for dynamic load type selection (`historical` or `manual`), passes parameters to child note-

books via `dbutils.notebook.run()`, logs each step’s execution status to the audit log table, implements a file-based lock mechanism to prevent concurrent runs, archives processed files after successful execution, and enforces a 7-day log retention policy via a scheduled DELETE statement.

Table 2.2: Weekly Work Summary — Zomato ETL Project

Week	Activity	Outcome
1	Environment Setup & Architecture Design	Unity Catalog hierarchy, Volume, folder structure, and all Delta table schemas defined and validated
2	Module 1 Development (JSON → CSV)	JSON parsing, 20-field extraction, filedate derivation, CSV generation, and archival logic implemented
3	Module 2 Development (Bronze Layer Load)	Incremental partition-level load strategy and dimension table refresh implemented and tested
4	Module 3 Development (Gold Layer Transformation)	All 7 business rules applied; summary table populated with audit columns and partition keys
5	Orchestration, Load Strategies & Validation	Driver Notebook built; manual and historical load notebooks developed; end-to-end pipeline validated

1.6 Solution Provided

The solution delivered is a fully modular, governance-first ETL pipeline on Databricks that transforms raw Zomato JSON restaurant data into an analytics-ready Delta Lake table through a structured, auditable, and operationally safe workflow. The following describes the key engineering decisions and their business rationale.

Unity Catalog as the Governance Backbone: All pipeline assets — catalogs, schemas, tables, and volumes — are provisioned and accessed exclusively through Unity Catalog (`zomato.etl`). This ensures centralized access control, table discovery, and metadata governance without relying on DBFS or unmanaged storage paths. Every `USE CATALOG` and `USE SCHEMA` statement is explicitly declared at the start of each notebook to prevent accidental writes outside the governed namespace.

Medallion Architecture for Data Quality Progression: The three-layer architecture enforces clear transformation lineage. The Bronze layer (`zomato_raw`) stores minimally transformed, schema-aligned raw data. The Gold layer (`zomato_summary`) applies all business rules and serves as the single source of truth for analytics. This separation allows safe reprocessing: if transformation logic changes, the Bronze layer remains intact and the Gold layer can be rebuilt without re-ingesting source files.

Incremental Load with Partition-Level Overwrite: Rather than truncating the entire `zomato_raw` table on each run, the pipeline identifies the specific `filedate` partitions in the incoming data, deletes only those partitions from the Delta table, and appends fresh records. This approach is idempotent — running the pipeline twice for the same date produces the same result — and computationally efficient, as unchanged historical partitions are never touched.

Computed Business Columns: Two derived columns add significant analytical value. The `m_rating_colour` column maps `aggregate_rating` to five colour-coded bands (Red, Amber, Light Green, Green, Gold) using PySpark’s `when/otherwise` construct, enabling intuitive rating-band filtering in BI dashboards. The `m_cuisines` column classifies restaurants as “Indian” or “World Cuisines” based on a keyword match on the `cuisines` field, supporting cuisine-category segmentation in reports.

Audit Columns and Execution Logging: Every record in `zomato_summary` carries a `create_datetime` timestamp and a `user_id` field that identifies the execution mode (`etl_user`, `manual_load`, or `historical_load`). Additionally, the `zomato_summary_log` table captures step-level metadata — job ID, step name, start/end timestamps, and status — for every pipeline execution, enabling SLA monitoring, failure diagnosis, and compliance auditing.

Lock Mechanism for Concurrency Safety: A file-based lock (`etl.lock`) is written to the Volume at the start of each Driver Notebook run and deleted in a `finally` block, guaranteeing release regardless of success or failure. If a second pipeline instance attempts to run while the lock exists, execution is immediately aborted with an exception. This prevents concurrent writes that could corrupt Delta table state.

Multiple Load Strategies: The pipeline supports three execution modes — *scheduled daily load* (driven by the Driver Notebook), *manual load* (filtered by `filedate` and/or `country_code` via Databricks widgets, appending targeted records), and *historical load* (full overwrite of `zomato_summary` using all available raw data, with `overwriteSchema=true` for schema evolution safety). This operational flexibility makes the pipeline suitable for both steady-state daily operations and one-time recovery or backfill scenarios.

As illustrated in Figure 2.2, the Driver Notebook executes each module sequentially, logging status at every step, while the Gold layer output presents country-enriched, rating-classified, and audit-stamped restaurant records partitioned for optimal query performance.

1.7 Results

The pipeline was successfully validated end-to-end across all three execution modes. The following outcomes were confirmed through systematic testing.

The `zomato_raw` Bronze table was correctly populated with partitioned raw records

Apr 17, 2026, 03:17 PM	Apr 17, 2026, 03:17 PM	...n/environment_setup	23s	✓ Succeeded
Apr 17, 2026, 03:18 PM	Apr 17, 2026, 03:18 PM	...module1_json_to_csv	26s	✓ Succeeded
Apr 17, 2026, 03:18 PM	Apr 17, 2026, 03:19 PM	...n/module2_raw_load	29s	✓ Succeeded
Apr 17, 2026, 03:19 PM	Apr 17, 2026, 03:19 PM	...ission/historical_load	15s	✓ Succeeded
<pre> Running Environment Setup Wrote 7 bytes. Starting Zomato ETL Pipeline Running Module 1 Running Module 2 Running Historical Load Archiving processed JSON files Files moved to archive Old logs cleaned Zomato ETL Pipeline Completed Successfully Lock Released </pre>				

Figure 2.2: Driver Notebook — Step-wise Execution and Logging

for each processed filedate, with schema enforcement preventing type mismatches during CSV ingestion. The `dim_country` dimension table was refreshed accurately, with country names correctly mapped to their respective integer codes. The `zomato_summary` Gold table reflected all applied business rules, with no NULL cuisine records, no duplicate restaurant–filedate combinations, correctly classified rating colours and cuisine categories, and fully populated audit columns.

Partition pruning was confirmed to be effective: queries filtered by `p_country_name` or `p_filedate` scanned only the relevant Delta partitions, significantly reducing compute overhead for country-specific or date-specific analytical workloads. The lock mechanism was tested by simulating a concurrent execution, which was correctly rejected with an exception and logged as a failed job in the audit table.

1.8 Conclusion

The Zomato ETL Pipeline project delivered a production-grade, end-to-end data engineering solution on the Databricks Lakehouse platform. By implementing the Medallion Architecture, Unity Catalog governance, Delta Lake ACID storage, and a modular notebook-based orchestration framework, the pipeline addressed all identified data quality, governance, and operational challenges associated with processing semi-structured Zomato restaurant metadata.

From an engineering standpoint, the project provided hands-on experience with real-world data platform design patterns — specifically, partition-level incremental loading, schema enforcement, computed column derivation, audit trail design, and concurrency control — all of which are foundational skills for a production data engineer. From a

Table 2.3: Pipeline Validation Test Cases — Zomato ETL

Test Case	Description	Expected Result	Status
TC-01	JSON parsing and 20-field extraction	All columns populated correctly	Pass
TC-02	Filedate derivation from file-name pattern	Correct date tagged per file	Pass
TC-03	NULL cuisine record filtering	Zero NULL cuisine rows in Gold table	Pass
TC-04	Deduplication on (restaurant_id, filedate)	No duplicate rows in summary	Pass
TC-05	m_rating_colour derivation	Correct band for each rating range	Pass
TC-06	m_cuisines classification	Indian / World Cuisines correctly assigned	Pass
TC-07	Incremental partition-level overwrite	Only target partitions refreshed	Pass
TC-08	Concurrent run lock prevention	Second run aborted; failure logged	Pass
TC-09	Manual load with filedate filter	Only specified date records loaded	Pass
TC-10	Historical load full overwrite	Summary table fully rebuilt; schema preserved	Pass

business standpoint, the resulting `zomato_summary` table enables Zomato’s analytics teams to perform country-wise restaurant benchmarking, cuisine-category segmentation, rating-band distribution analysis, and pricing tier insights with high confidence in data quality and completeness.

The multi-load-strategy architecture (scheduled, manual, and historical) ensures that the pipeline is operationally resilient, capable of supporting not just routine daily processing but also targeted backfills and full dataset recoveries without disrupting the downstream analytical environment. This project established the foundational data engineering competencies applied in the subsequent retail and pharmaceutical pipeline projects undertaken during the internship.

Project 2: RetailPulse Analytics Platform — End-to-End Retail Data Engineering on AWS

2.1 Introduction

Modern retail and e-commerce organizations generate vast volumes of transactional data every second — spanning customer purchases, inventory movements, partner fulfillments, and payment events — across multiple systems, formats, and geographies. Transform-

Table 2.4: Technology Stack — Zomato ETL Pipeline

Sr. No.	Technology / Tool	Role in Pipeline
1	Databricks Free Edition	ETL execution and notebook orchestration platform
2	Unity Catalog	Centralized governance, schema, and table management
3	Delta Lake	ACID-compliant storage for Bronze, Dimension, and Gold tables
4	Databricks Volumes	Managed file storage for raw, staged, CSV, and archive data
5	PySpark & Spark SQL	Data transformation, filtering, joining, and aggregation
6	Python (dbutils, re, uuid)	Orchestration utilities, filename parsing, job identification

ing this raw, heterogeneous data into reliable, analytics-ready insights demands a scalable, governed, and automated cloud-native data platform. Without such infrastructure, organizations face inaccurate revenue reporting, inventory stockouts, delayed decision-making, and competitive disadvantage.

This project, **RetailPulse Analytics Platform**, was undertaken during the internship at Infocepts as a comprehensive data engineering engagement simulating how modern retail enterprises architect and operate end-to-end data platforms on **Amazon Web Services (AWS)**. The platform ingests synthetic retail data from multiple sources — a relational database (Amazon RDS MySQL), semi-structured external partner JSON feeds, and flat-file inventory exports — processes it through a distributed **Apache Spark ETL pipeline on Amazon EMR**, stores validated and enriched data in a layered **Amazon S3 Data Lake**, enables serverless SQL analytics via **Amazon Athena** over a **AWS Glue Data Catalog**, delivers business intelligence through **Amazon QuickSight** dashboards, and operates a fully automated real-time **alerting system** using **AWS Lambda**, **Amazon SNS**, and **Amazon SQS**.

The platform is designed around five core principles: scalability, data quality, multi-source integration, actionable analytics, and operational automation. Together, these principles mirror the architecture patterns employed by large-scale retail data platforms in production environments.

2.2 Problem Identification

The RetailPulse platform was designed to address a cluster of interconnected data engineering challenges that are representative of real-world retail data systems.

(i) Data Quality Degradation: Raw transactional order data contains four categories of quality defects that were deliberately injected during synthetic data generation to simulate real-world conditions. *Duplicate records* arise when the same `order_id` is ingested multiple times across pipeline reruns or source re-extractions, causing double-counting of revenue metrics. *Missing critical fields* — specifically NULL values in `order_id` and `customer_id` — break downstream joins and customer-level aggregations. *Future-dated transactions* (orders with `order_date` beyond the current timestamp) distort time-series trend analyses and daily revenue reports. *Invalid pricing records* — where the `unit_price_inr` exceeds `mrp_inr` — produce negative or misleading discount percentages and incorrect margin calculations.

(ii) Multi-Source Schema Heterogeneity: The platform ingests data from two structurally different sources. Internal RDS records use fields named `order_id`, `customer_id`, and `unit_price`, while external partner JSON feeds use `order_ref`, `cust_id`, and `price` for the same semantic attributes. Without explicit schema normalization, these sources cannot be unioned into a single analytical dataset, rendering cross-source reporting impossible.

(iii) Absence of Real-Time Monitoring: Even after data is cleaned and loaded into the analytical layer, there is no automated mechanism to detect critical business events — such as top-seller stockouts, payment-method-specific cancellation spikes, or zero-order categories. Business users are required to manually inspect dashboards, which introduces latency in operational response.

(iv) Lack of Cost-Optimized Query Infrastructure: Storing processed data in non-columnar, non-partitioned formats on S3 leads to full-dataset scans in Athena, significantly increasing query costs and execution times for date-range or category-filtered analytical workloads.

(v) No Referential Integrity Enforcement: Orders referencing `customer_id` values absent from the customer master table — termed “orphan orders” — are neither rejected nor flagged, causing silently broken joins and misleading customer-level metrics in downstream analytics.

2.3 Objectives

The RetailPulse Analytics Platform was designed with the following engineering and business objectives:

- Build a **scalable, layered S3 Data Lake** with clearly separated Raw, Processed,

Quarantine, and Aggregates zones to support both data lineage and reprocessing flexibility.

- Implement a **multi-source ingestion pipeline** that reads structured data from Amazon RDS via JDBC and semi-structured partner data from S3-hosted JSON files, normalizing both into a unified schema using PySpark on EMR.
- Enforce **four data quality validation rules** in Spark — null key detection, future-date filtering, duplicate deduplication using window functions, and orphan customer flagging — routing invalid records to typed quarantine sub-zones in S3 for governance and investigation.
- **Enrich validated order records** with computed business metrics including order revenue, discount percentage, profit margin, and weekend transaction flag, producing a `fact_orders` table optimized for analytics.
- Store processed data in **Parquet format partitioned by order_date only** on S3, enabling partition-pruned Athena queries for cost-efficient date-range analytics.
- Provision a **Glue Data Catalog** using automated crawlers to maintain schema metadata for the processed fact table, enabling seamless Athena SQL querying without manual DDL management.
- Deliver **five QuickSight dashboards** covering daily sales trends, product ranking, category revenue breakdown, order status distribution, and geographic revenue visualization using SPICE in-memory caching.
- Implement a **serverless alerting pipeline** using AWS Lambda, SNS, SQS, and RDS to automatically detect and notify stakeholders of critical business anomalies — inventory stockouts, high cancellation rates, and zero-order categories — on a scheduled basis.

2.4 Architecture Overview

The RetailPulse platform is structured as a seven-stage, cloud-native data engineering system on AWS, spanning data generation through automated alerting. Figure 2.3 illustrates the high-level architecture.

The architecture flows through the following layers. **Stage 1 — Data Generation** uses Python scripts with the Pandas, Faker, NumPy, and pytz libraries to synthesize six datasets: customers, products, distributors, orders (with intentional quality defects), inventory snapshots, and external partner orders in JSON format, simulating realistic Indian

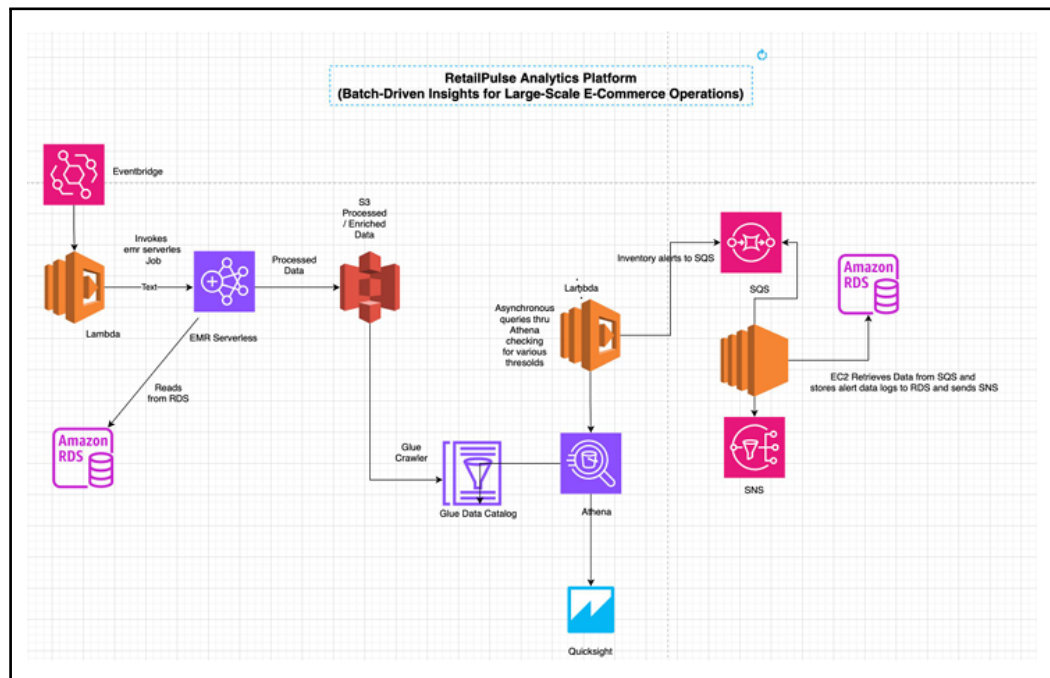


Figure 2.3: RetailPulse Analytics Platform — End-to-End AWS Architecture

retail operations at scale. **Stage 2 — S3 Data Lake** establishes the central storage backbone, with the bucket `retailpulse-data-lake` organized into four zones: `raw/` (original unmodified files), `processed/` (clean Parquet datasets), `quarantine/` (rejected records with typed sub-folders for null key, future date, and duplicate violations), and `aggregates/` (precomputed customer and product metrics). **Stage 3 — Amazon RDS** provides structured relational storage for the core transactional tables, loaded from S3 CSVs using MySQL's `LOAD DATA LOCAL INFILE` command, and serves as the primary JDBC data source for the Spark ETL job. **Stage 4 — Amazon EMR (Spark ETL)** is the core transformation engine, reading from both RDS and S3 JSON, normalizing schemas, applying all four validation rules, enriching records with business metrics, and writing the output as date-partitioned Parquet to `processed/facts/orders/`. **Stage 5 — AWS Glue and Amazon Athena** form the metadata and query layers, where an automated Glue crawler infers the schema of the processed fact table and registers it in the Glue Data Catalog, enabling serverless SQL analytics through Athena with full partition pruning support. **Stage 6 — Amazon QuickSight** provides the visualization layer, consuming Athena datasets via SPICE for interactive business dashboards. **Stage 7 — Serverless Alerting** connects Lambda, SNS, SQS, and RDS into an automated monitoring pipeline triggered on a CloudWatch schedule.

Table 2.5: AWS Services Used — RetailPulse Analytics Platform

Stage	AWS Service / Tool	Role in Platform
1	Python (Pandas, Faker, NumPy, pytz)	Synthetic data generation with quality defect injection
2	Amazon S3	Layered data lake (Raw, Processed, Quarantine, Aggregates)
3	Amazon RDS (MySQL 8.x)	Structured relational source; JDBC ingestion point for Spark
4	Amazon EMR (Apache Spark)	Distributed ETL: validation, schema unification, enrichment
5	AWS Glue Data Catalog	Automated schema discovery and metadata management
5	Amazon Athena	Serverless SQL analytics with partition pruning on S3 Parquet
6	Amazon QuickSight (SPICE)	Interactive BI dashboards for business stakeholders
7	AWS Lambda	Scheduled execution of Athena monitoring queries
7	Amazon SNS	Email notification delivery to alert subscribers
7	Amazon SQS	Reliable message queuing for alert fault tolerance

2.5 Work Carried Out

The internship contribution spanned all seven stages of the platform lifecycle. The work was executed sequentially, with each stage building directly on the outputs of the previous.

Stage 1 — Synthetic Data Generation: Six datasets were generated using Python scripts. The customer dataset (1,000 records) includes demographic attributes — city, state, pincode, customer segment, loyalty tier, acquisition channel, and date of birth — generated using `Faker('en-IN')` to ensure Indian retail context. The product catalog (200 products) covers categories, sub-categories, brands, MRP, cost price, and reorder thresholds. The orders dataset (5,000+ records) was generated with intentional quality defects: approximately 5% of records have `NULL customer_id`, 3% have future `order_date` values, and 4% are exact duplicates. The distributor, inventory snapshot, and partner JSON datasets complete the multi-source simulation. All files were exported as CSV or JSON using Pandas.

Stage 2 — S3 Data Lake Configuration: An S3 bucket (`retailpulse-data-lake`) was provisioned in the `ap-south-1` (Mumbai) region using the AWS CLI. A four-zone folder hierarchy was established: `raw/rds/` and `raw/partner_json/` for landing zone uploads, `processed/facts/` and `processed/dimensions/` for clean Parquet output, `quarantine/null_key/`, `quarantine/future_date/`, and `quarantine/duplicate/` for typed invalid record routing, and `aggregates/` for precomputed customer and prod-

uct metric tables. All CSV and JSON files were uploaded to the raw zone using `aws s3 cp` commands.

Stage 3 — Amazon RDS Setup and Schema Loading: A MySQL 8.x RDS instance (`retailpulse-db`, `db.t3.micro`) was provisioned with a VPC security group configured to allow inbound MySQL traffic (port 3306) from the EMR cluster's EC2 nodes. Five tables — `customers`, `products`, `distributors`, `orders`, and `inventory_snapshot` — were created with explicit DDL schemas enforcing data types, primary keys, and column constraints. Data was loaded from S3 CSVs using MySQL's `LOAD DATA LOCAL INFILE` command after downloading files from S3 to the EC2 bastion host.

Stage 4 — EMR Spark ETL Pipeline: A PySpark application was submitted to an EMR cluster. The Spark session was initialized with optimized shuffle partition configuration (`spark.sql.shuffle.partitions=200`) to balance parallelism. The `orders` table was read from RDS using JDBC with the MySQL Connector, and partner JSON data was read directly from S3. Schema normalization was applied to the partner `DataFrame` — renaming `order_ref` to `order_id`, `cust_id` to `customer_id`, `item_id` to `product_id`, and `price` to `unit_price` — before unioning both sources using `unionByName(allowMissingColumns=True)`. Four sequential validation rules were enforced: (i) NULL key detection using column-level `isNull()` checks with subtraction-based separation of invalid records; (ii) future-date filtering comparing `order_date` against `current_timestamp()`; (iii) deduplication using `row_number()` over a window partitioned by `order_id` ordered by `order_date`, retaining only the first occurrence; and (iv) orphan customer detection via a left join against the `customers` `DataFrame` with missing matches flagged rather than discarded. Invalid records from each validation step were written to their respective quarantine sub-zones as Parquet. Four enrichment columns were then derived: `revenue` as `quantity * unit_price`, `discount_pct` as `(mrp - unit_price) / mrp * 100`, `is_weekend` using `dayofweek()` with values 1 and 7 mapped to `True`, and `profit_margin` as `revenue - cost_price_inr`. The enriched `DataFrame` was written to `processed/facts/orders/` as date-partitioned Parquet using `partitionBy("order_date_only")`.

Stage 5 — Glue Crawler and Athena Analytics: A Glue crawler was configured to scan the `processed/facts/orders/` S3 path, infer the Parquet schema and partition structure, and register a `fact_orders` table in the `retailpulse_analytics` Glue database. Four advanced Athena SQL queries were developed and validated: a daily revenue trend query using `LAG()` window functions to compute day-over-day revenue growth percentage; a product ranking query using `RANK()` over revenue aggregations to identify top-ten products; a customer cohort analysis query grouping customers by first order month; and an inventory risk analysis query using a `CASE` expression to flag products where average closing stock falls below average reorder threshold.

Stage 6 — QuickSight Dashboards: QuickSight was connected to the Glue Data Catalog and Athena. Five dashboards were built using SPICE-imported datasets: Daily Sales Summary (line chart for revenue trend, bar chart for order volume, KPI cards for average order value), Top Products Analysis (horizontal bar chart for top-ten revenue products), Revenue by Category (pie and stacked bar charts), Order Status Breakdown (donut chart and status trend lines), and a Geographic Revenue Map (bubble map by customer pincode).

Stage 7 — Lambda Alerting System: An SQS standard queue (`retailpulse-alert-queue`) and SNS topic (`retailpulse-alert-topic`) with email subscriptions were provisioned. An IAM role with `AmazonAthenaFullAccess`, `AmazonSNSFullAccess`, and `AmazonSQSFullAccess` policies was assigned to the Lambda function. The Lambda handler was implemented in Python using the `boto3` SDK to execute Athena monitoring queries, evaluate business rule conditions, and publish concurrent alert messages to SNS and SQS. An RDS `alerts_log` table persists all alert records for historical audit. The function is triggered on a CloudWatch Events schedule (every 5 minutes) to provide near-real-time anomaly detection.

Table 2.6: Weekly Work Summary — RetailPulse Analytics Platform

Week	Activity	Outcome
1	Data Generation & S3 Data Lake Setup	Six synthetic datasets generated with quality defects; S3 four-zone structure provisioned and populated
2	RDS Schema Design & Data Loading	Five MySQL tables created with enforced DDL; data loaded via <code>LOAD DATA LOCAL INFILE</code> ; JDBC connectivity verified
3	EMR Spark ETL — Ingestion, Validation & Enrichment	Multi-source unification, four validation rules enforced, quarantine zones populated, enrichment columns derived
4	Glue Catalog, Athena Analytics & QuickSight Dashboards	Crawler registered fact table; four advanced SQL analyses validated; five QuickSight dashboards built on SPICE
5	Lambda Alerting System & End-to-End Validation	Four alert types implemented; SNS/SQS/RDS pipeline tested; full platform validated end-to-end

2.6 Solution Provided

The RetailPulse Analytics Platform delivers a complete, production-pattern AWS data engineering solution. The following engineering decisions and their business rationale constitute the core of the solution.

Layered S3 Data Lake with Typed Quarantine Zones: Rather than applying a binary accept/reject pattern, the pipeline routes invalid records to semantically named quarantine

sub-folders (`null_key/`, `future_date/`, `duplicates/`) as Parquet files. This enables the data engineering team to analyze rejection patterns, quantify the volume of each defect type over time, and remediate upstream data quality issues systematically. The raw zone is never modified after initial upload, preserving a tamper-proof audit trail for reprocessing.

Schema Normalization via Column Renaming and `unionByName()`: Partner JSON data uses different field names from the internal RDS schema. Rather than accepting schema mismatch silently, the pipeline explicitly renames all misaligned columns before union, ensuring semantic correctness. The `allowMissingColumns=True` parameter in `unionByName()` prevents pipeline failure when partner data lacks optional fields present in the internal schema, providing robustness against evolving partner data contracts.

Window Function–Based Deduplication: Duplicate orders are identified using `row_number()` over a window partitioned by `order_id` and ordered by `order_date` (earliest-first), retaining only the chronologically first occurrence. This approach is deterministic and produces consistent results regardless of the physical order of rows in the input DataFrame, making it safe for parallel distributed execution on EMR.

Parquet + Date Partitioning for Cost-Optimized Athena Queries: Processed data is stored in columnar Parquet format partitioned by `order_date_only`. Athena’s partition pruning ensures that a query filtered to a specific date range scans only the corresponding partition directories rather than the full dataset. For a dataset spanning 365 days, a 7-day query window reduces the scan footprint by approximately 98%, directly lowering Athena per-query cost (billed at \$5 per TB scanned).

Four Business Enrichment Metrics: The `revenue` column (`quantity * unit_price`) serves as the primary measure in all sales aggregations. The `discount_pct` column enables promotional effectiveness analysis by quantifying the markdown depth per transaction. The `is_weekend` boolean flag supports behavioural segmentation by distinguishing weekday versus weekend purchase patterns. The `profit_margin` column (`revenue - cost_price_inr`) provides the basis for gross margin trend analysis directly in the fact table, eliminating the need for a separate cost dimension join during reporting.

Four Athena Analytics Capabilities: The revenue trend query employs the `LAG()` SQL window function to compute day-over-day revenue growth percentage without requiring application-side data manipulation. The product ranking query uses `RANK()` over revenue aggregations to generate a live leaderboard. The inventory risk analysis uses a conditional CASE expression comparing average closing stock against the reorder threshold, producing an `AT_RISK / STABLE` status flag per product that directly feeds the Lambda alerting logic.

Fault-Tolerant Serverless Alerting: The Lambda–SNS–SQS triad ensures no alert is

lost even under transient system failures. SNS provides instant multi-subscriber email delivery; SQS buffers alert messages with configurable retention and enables retry on downstream failures; and the RDS `alerts_log` table maintains a persistent historical record of all alerts for trend analysis and recurring anomaly detection. Four distinct alert types — top-seller stockout, high payment-method cancellation rate exceeding 25%, zero-order category detection, and significant day-over-day revenue drop — cover the most operationally critical retail business events. As shown in Figure 2.4, the Spark ETL stage routes records through sequential validation gates before enrichment, while the QuickSight layer exposes the resulting clean metrics through interactive, stakeholder-ready dashboards.

2.7 Results

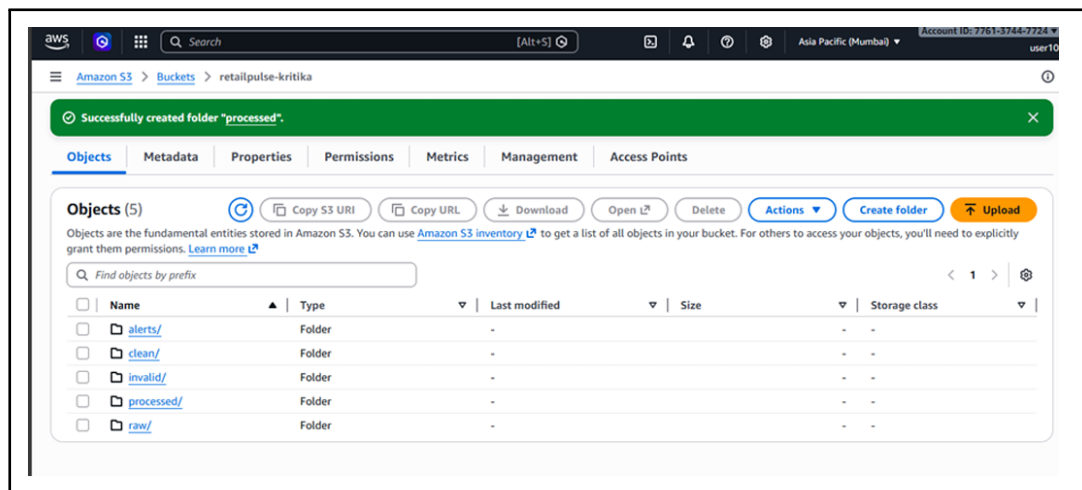
The platform was validated end-to-end across all seven stages. The following outcomes were confirmed through systematic testing and query-level verification.

The S3 Data Lake correctly partitioned output into all four zones. Quarantine sub-zones received the expected volume of rejected records: NULL key records, future-dated orders, and detected duplicates were each written to their respective sub-folders as Parquet and verified by row count via Athena. The `fact_orders` table in Athena reflected zero NULL critical key values, zero future-dated records, and zero duplicate `order_id` entries in the processed zone. All four enrichment columns (`revenue`, `discount_pct`, `is_weekend`, `profit_margin`) were correctly computed and present in the Parquet schema.

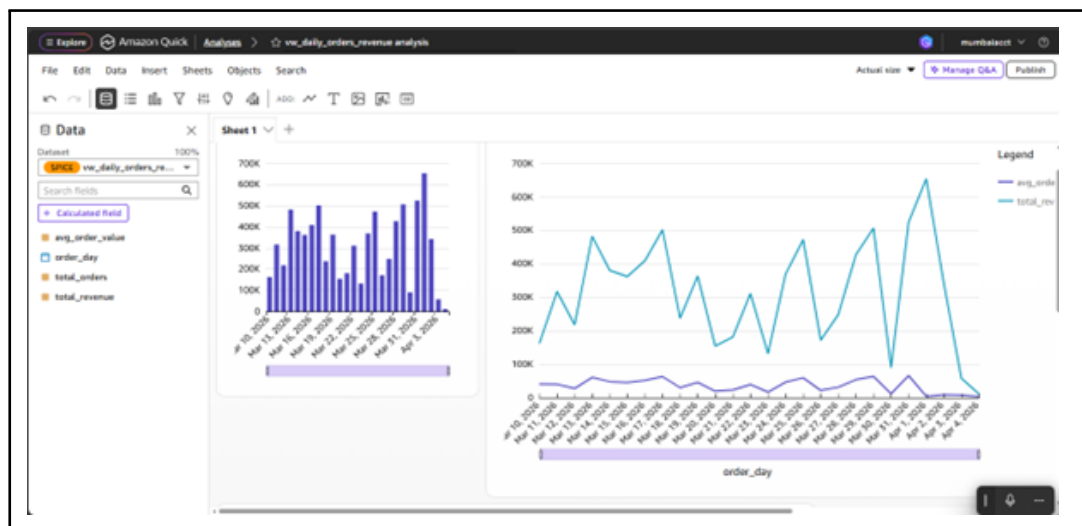
Partition pruning was confirmed to be effective: a 7-day date-range Athena query on a 90-day dataset reduced data scanned from the full dataset size to approximately one-thirteenth, consistent with the expected partition pruning ratio. The four advanced Athena SQL queries executed successfully, returning revenue trend percentages, top-ten product rankings with margin data, cohort retention counts, and inventory risk classifications. QuickSight SPICE dataset ingestion succeeded across all five dashboard report types. The Lambda alerting function correctly identified seeded anomalies in the test dataset, dispatched email notifications via SNS, enqueued messages in SQS, and persisted alert records to the RDS log table.

2.8 Conclusion

The RetailPulse Analytics Platform project successfully delivered a complete, seven-stage, production-pattern data engineering system on AWS, addressing the multi-dimensional challenges of data quality, multi-source integration, scalable storage, cost-optimized analytics, business intelligence delivery, and automated operational monitoring in a retail context.



(a) S3 bucket — Validation & Quarantine Routing



(b) QuickSight Dashboard — Daily Sales Summary and Revenue by Category

Figure 2.4: RetailPulse Platform — Spark Processing and BI Output

Table 2.7: Platform Validation Test Cases — RetailPulse Analytics Platform

Test Case	Description	Expected Outcome	Status
TC-01	NULL <code>order_id</code> / <code>customer_id</code> detection	Records routed to quarantine/null_key/	Pass
TC-02	Future-dated order filtering	Records routed to quarantine/future_date/	Pass
TC-03	Duplicate order deduplication via window function	Only earliest occurrence retained	Pass
TC-04	Orphan customer flagging via left join	Orders with missing customer flagged, not dropped	Pass
TC-05	Schema normalization (partner JSON)	Renamed columns correctly unioned with RDS data	Pass
TC-06	Revenue and discount enrichment columns	Mathematically correct values for all records	Pass
TC-07	Parquet date-partition pruning in Athena	7-day query scans only 7 partitions	Pass
TC-08	Glue crawler schema inference	<code>fact_orders</code> table correctly registered	Pass
TC-09	QuickSight SPICE dataset ingestion	All five dashboards rendered without error	Pass
TC-10	Lambda alert — high cancellation rate	SNS email sent; SQS message enqueued; RDS log written	Pass
TC-11	Lambda alert — top-seller stockout	Alert generated for product with revenue rank 1 and zero stock	Pass
TC-12	Lambda alert — zero-order category	Alert generated for category with no orders today	Pass

From an engineering perspective, the project provided deep, applied experience with a broad spectrum of AWS cloud-native services — EMR, RDS, S3, Glue, Athena, QuickSight, Lambda, SNS, and SQS — and with core distributed processing patterns in PySpark including window-function deduplication, schema normalization via column renaming and `unionByName()`, quarantine-based data quality governance, and columnar partitioned storage optimization. These competencies are directly transferable to enterprise retail data platform roles.

From a business perspective, the platform establishes the data infrastructure necessary for a retail organization to move from reactive, manually-inspected reporting to proactive, automated data-driven operations. The QuickSight dashboards provide real-time visibility into sales performance, product economics, and customer geography, while the Lambda alerting system ensures that critical operational events — stockouts, payment-driven cancellation surges, and demand anomalies — are surfaced to the responsible teams within

minutes of occurrence rather than hours or days. The architecture is inherently scalable: by increasing the EMR cluster size, adjusting Spark partition counts, and adding Glue crawler schedules, the same platform can accommodate order-of-magnitude growth in data volume without structural change.

Project 3: Pharmaceutical Supply Chain Streaming Analytics Platform

3.1 Introduction

Pharmaceutical supply chains operate under uniquely stringent regulatory, quality, and operational constraints that conventional supply chain platforms are not architected to handle. Temperature-sensitive medicines — including vaccines, insulin, biologics, and controlled substances — must maintain verifiable cold chain integrity from the point of manufacture through storage, distribution, and final dispensing. Every batch produced must carry a complete, auditable lineage connecting manufacturing records, quality control test results, distributor inventory movements, and pharmacy sales transactions. When a quality failure occurs post-release, the organization must immediately quantify the recall impact — identifying affected orders, in-transit inventory, and revenue exposure — within hours, not days.

Traditional pharmaceutical data systems fail to meet these requirements because they operate as siloed, batch-refresh reporting platforms. Temperature excursions go undetected for hours; recall impact assessment is performed manually via spreadsheets and cross-departmental calls; inventory visibility is fragmented across distributor warehouses; and QC failure data is not automatically correlated with live order fulfillment status.

This project, undertaken during the internship at Infocepts, delivers an **end-to-end, event-driven pharmaceutical supply chain analytics platform** that addresses these challenges through a tightly integrated combination of **real-time streaming, batch ETL pipelines, and unified data modeling**. The platform continuously ingests temperature sensor readings, pharmacy sales transactions, and distributor inventory change events via **Amazon Kinesis Data Streams**, processes them in real time using **Spark Structured Streaming on Amazon EMR**, persists validated data to a **Hive-managed S3 data lake**, and correlates all event streams through a batch-centric data spine built around a common `batch_id` identifier. Batch-layer data — including the manufacturing batch master, product master, inventory snapshots, and QC test results — is extracted from Amazon RDS via **Apache Sqoop** into the Hive warehouse and processed through raw, clean, valid, and invalid table tiers. The final analytical layer is served through **Amazon Athena** and visualized via **Amazon QuickSight** dashboards, enabling both operational and executive stakeholders to consume real-time intelligence about cold chain compliance, inventory risk, batch traceability, and recall impact.

3.2 Problem Identification

The pharmaceutical supply chain analytics platform was designed in direct response to four well-defined business and engineering problems that existing systems failed to resolve.

(i) Cold Chain Integrity Assurance Without Real-Time Detection: Temperature-sensitive pharmaceutical products such as vaccines and biologics must be stored and transported within batch-specific temperature ranges (e.g., 2–8C for cold chain products). IoT sensors mounted on storage units and transport vehicles emit temperature readings every 30 seconds. Without a real-time streaming pipeline, excursions are detected only during periodic manual audits — often hours after the breach — by which time the affected product may have already been dispensed to patients or moved to the next distribution node. Regulatory bodies require documented, near-real-time evidence of continuous cold chain compliance.

(ii) Absence of Batch-Level End-to-End Traceability: Every unit dispensed at a pharmacy must be traceable back to its manufactured batch, including associated QC test results, storage temperature history, and supply chain movement events. In existing setups, these records exist in four different systems — the manufacturing ERP, the LIMS (Laboratory Information Management System), the distributor WMS, and the pharmacy POS — with no unified linking mechanism. The `batch_id` is the natural key for this linkage, but no platform previously used it as the central spine across all data domains.

(iii) Fragmented Distributor Inventory Visibility and Delayed Replenishment: Inventory data is siloed across multiple distributors and warehouses. The absence of a unified, real-time inventory view prevents early identification of stockout risks. Without computing live `days_of_stock` coverage based on current stock levels and historical sales velocity, replenishment orders are triggered reactively rather than proactively, leading to unmet patient demand and avoidable write-offs for near-expiry stock.

(iv) Manual, Delayed QC Failure Impact Assessment: When a batch fails post-release quality testing — identified via abnormal `potency_pct`, `sterility_result`, or `endotoxin_result` values in the LIMS — operations teams must manually identify which orders included units from that batch, which distributor nodes currently hold that batch's inventory, and what the total revenue exposure is. This process is typically manual, error-prone, and takes 24–72 hours — during which unsafe products may continue moving through the supply chain.

3.3 Objectives

The pharmaceutical supply chain analytics platform was designed with the following engineering and compliance objectives:

- Implement a **real-time streaming ingestion pipeline** using Amazon Kinesis Data

Streams and Spark Structured Streaming on Amazon EMR to continuously consume temperature sensor events, pharmacy sales transactions, and distributor inventory change events with sub-minute latency.

- Apply **stream-level data quality validation** — including temperature unit normalization (Fahrenheit to Celsius conversion), physical range validation (`temperature_c` within -50 to 100°C), sales quantity and price positivity checks, and inventory change type whitelisting — routing valid records to Hive-managed Parquet tables and discarding invalid events.
- Build a **batch ETL pipeline** using Apache Sqoop to extract the manufacturing batch master, product master, QC test results, and inventory snapshot tables from Amazon RDS MySQL into the Hadoop/Hive ecosystem, processing each dataset through four table tiers: raw, clean, valid, and invalid.
- Establish a **unified batch-centric data spine** by using `batch_id` as the common linking key across all datasets — temperature readings, sales transactions, inventory movements, QC results, and the batch master — enabling complete end-to-end product lifecycle traceability.
- Enable **near-instant recall impact analysis** by cross-referencing QC failure records against live order fulfillment status and distributor inventory to produce an automated recall impact list within minutes of a batch failure determination.
- Deliver six **Athena-powered analytical queries** covering high-value batch cold chain compliance status, regional stockout risk, recall sales impact, repeat temperature violation distributors, unusual pharmacy purchase patterns, and active batch QC and temperature compliance posture.
- Build **QuickSight dashboards** connected to the Athena analytical layer via SPICE, delivering cold chain compliance, regional sales performance, and inventory risk visualizations to operational and executive stakeholders.

3.4 Architecture Overview

The platform is structured across two parallel processing lanes — a real-time streaming lane and a batch processing lane — that converge at the Hive metastore and are jointly served through Amazon Athena and QuickSight. Figure 2.5 illustrates the end-to-end architecture.

Streaming Lane: Three event streams — temperature sensor readings, pharmacy sales transactions, and distributor inventory change events — are produced by a Python

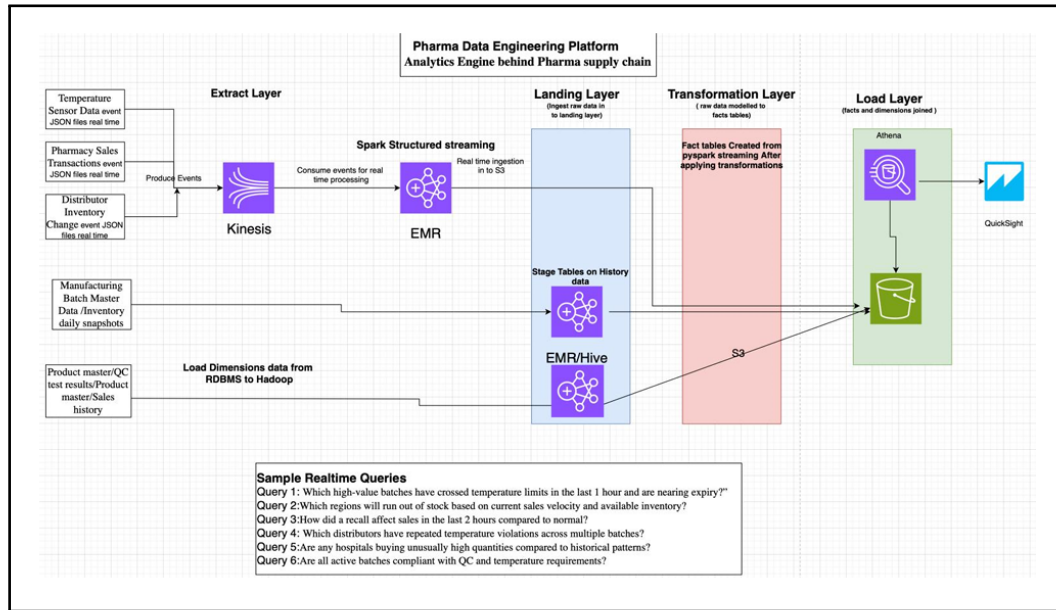


Figure 2.5: Pharmaceutical Supply Chain Analytics Platform — Kinesis + Spark Architecture

producer.py script executing on an EC2 instance (t3.micro, Amazon Linux 2023) in the ap-south-1 region. The producer reads from three CSV datasets, assigns a source_type tag to each record, shuffles the combined dataset, and publishes records to a single Kinesis Data Stream (pharma_kinesis_kritika) with a 30-second inter-record delay using the boto3 Kinesis client. On the consumer side, a PySpark Structured Streaming application (consumer.py) runs on the EMR Hadoop node, reading from Kinesis using the spark-sql-kinesis connector JAR (spark-sql-kinesis-2.12-1.2.0-sp submitted via spark-submit. The consumer defines a superset schema covering all three event types, parses JSON records using from_json(), splits the unified stream into three typed DataFrames by source_type filter, applies stream-specific validation and cleaning logic, and writes three persistent Parquet streams to the pharma_streaming_db Hive database with 120-second micro-batch triggers and checkpoint locations on S3.

Batch Lane: Four batch datasets — manufacturing batch master, product master, QC test results, and daily inventory snapshots — reside in Amazon RDS MySQL. These are extracted to HDFS using sqoop import with the MySQL Connector/J 8.0.22 JDBC driver and stored as textfile format. In Hive, each dataset is processed through four table tiers: raw (external table over HDFS textfile), clean (type-cast, trimmed, null-handled CTAS), valid (business-rule-filtered), and invalid (complement set for governance). The valid batch tables are then exported to S3 as Parquet using Hive CTAS with STORED AS PARQUET, making them available to Athena via the Glue Data Catalog.

Analytical Layer: Amazon Athena queries the S3-backed Parquet tables through the

pharma_db_kritika database in the Glue catalog. Six complex analytical queries join streaming and batch data via `batch_id` to produce cold chain compliance reports, recall impact analyses, and inventory risk assessments. QuickSight consumes these query results via SPICE-imported datasets for dashboard delivery.

Table 2.8: Data Sources and Schemas — Pharmaceutical Supply Chain Platform

Source	Dataset	Transport	Key Fields
Streaming	Temperature Sensor Events	Kinesis (every 30 sec)	<code>sensor_id,</code> <code>batch_id,</code> <code>temperature_c,</code> <code>alert_flag</code>
Streaming	Pharmacy Sales Transactions	Kinesis (real-time)	<code>transaction_id,</code> <code>batch_id,</code> <code>revenue_inr,</code> <code>fulfillment_status</code>
Streaming	Distributor Inventory Changes	Kinesis (real-time)	<code>event_id,</code> <code>batch_id,</code> <code>change_type,</code> <code>current_stock</code>
Batch	Manufacturing Batch Master	JDBC to S3 (daily 02:00)	<code>batch_id,</code> <code>min_temp_c,</code> <code>max_temp_c,</code> <code>batch_status</code>
Batch	Product Master	RDS MDM (daily 01:00)	<code>product_sku,</code> <code>cold_chain_required,</code> <code>controlled_flag</code>
Batch	QC Test Results	LIMS (daily incremental)	<code>qc_id, batch_id,</code> <code>overall_result,</code> <code>potency_pct</code>
Batch	Daily Inventory Snapshot	WMS JDBC (daily 03:00)	<code>snapshot_date,</code> <code>distributor_id,</code> <code>closing_stock,</code> <code>reorder_point</code>

3.5 Work Carried Out

The internship contribution covered the complete implementation of both the streaming and batch pipeline lanes, from Kinesis stream provisioning through Athena analytics and QuickSight visualization.

Phase 1 — Kinesis Stream Provisioning and EC2 Producer Setup: An Amazon Kinesis Data Stream (`pharma_kinesis_kritika`) was provisioned in `ap-south-1` in on-demand capacity mode to handle variable throughput without manual shard management. An EC2 instance (`t3.micro`, Amazon Linux 2023) was launched and con-

figured with Python 3, boto3, and pandas via `pip install`. AWS CLI was configured on the instance with access credentials and the `ap-south-1` region. Three CSV datasets — `temperature_events.csv`, `sales_transactions.csv`, and `inventory_events.csv` — were uploaded to the instance via SCP. The `producer.py` script was written to load all three datasets, tag each row with a `source_type` field, concatenate and shuffle the combined DataFrame, and publish records one by one to the Kinesis stream using `kinesis.put_record()` with a random integer partition key and a 30-second `time.sleep()` delay between records. Successful streaming was confirmed by terminal output showing sequential “*Sending record N from {source_type}*” messages.

Phase 2 — EMR Spark Structured Streaming Consumer: The Kinesis SQL connector JAR (`spark-sql-kinesis_2.12-1.2.0_spark-3.0.jar`) was uploaded to the EMR Hadoop master node via SCP. The initial `consumer.py` was written to initialize a `SparkSession` with `enableHiveSupport()` and `spark.sql.shuffle.partitions` read the Kinesis stream using `spark.readStream.format("aws-kinesis")` with `startingPosition=LATEST`, convert the binary payload to string using `selectExpr("CAST (AS STRING) ")`, and parse the JSON using `from_json()` with a manually defined superset schema covering all fields from all three event types. The parsed DataFrame was split into three typed streams by filtering on `source_type`.

Three stream-specific cleaning transformations were applied. The temperature stream cast raw string fields to Double, converted Fahrenheit readings to Celsius using a `when/otherwise` expression on the `unit` column, and filtered records outside the physical validity range (−50C to 100C). The sales stream cast `quantity_sold` to Integer and `unit_price`, `discount_pct`, and `revenue` to Double, then filtered records with non-positive quantity or price. The inventory stream cast `quantity_delta` and `current_stock` to Integer and filtered records whose `change_type` was not in the whitelist {RECEIPT, DISPATCH, RETURN, ADJUSTMENT}. A `partition_date` column was derived from `event_time` using `to_date()` for S3 Parquet partitioning. All three streams were written to Hive-managed Parquet tables (`pharma_streaming_db.temperature_streaming`, `pharma_streaming_db.sales_streaming`, `pharma_streaming_db.inventory_streaming`) using `writeStream .toTable()` with S3 checkpoint locations and 120-second micro-batch triggers. The Spark database and all three tables were pre-created using `spark-sql` with explicit DDL schemas, `USING PARQUET, PARTITIONED BY (partition_date)`, and `LOCATION` clauses pointing to the S3 bucket (`s3://pharma-data-lake-kritika/streaming`). The consumer was submitted to the EMR cluster using `spark-submit --jars spark-sql-kinesis-connector.jar consumer.py`. Successful data receipt was confirmed via `SELECT * FROM inventory_streaming` in `spark-sql`, returning correctly typed and partitioned inventory records.

Phase 3 — Batch Pipeline: Sqoop Extraction and Hive Processing: The MySQL Connector/J 8.0.22 JAR was uploaded to the EMR Hadoop node, moved to `/usr/lib/sqoop/lib/`

and permissions were set with `chmod 644`. Two Sqoop import commands were executed: one for `qc_results` and one for `product_master`, connecting via JDBC to the RDS endpoint, using `--as-textfile`, `--fields-terminated-by ' , '`, and `--lines-terminated-by '\n'` with `-m 1` (single mapper). The Hive database `pharma_db_kritika` was created and set as the active database. External tables (`product_master`, `qc_results_raw`, `inventory_snapshots_raw`) were created using `ROW FORMAT DELIMITED FIELDS TERMINATED BY ' , '` pointing to the HDFS extraction target directories. Clean tables were derived using Hive CTAS with explicit `CAST()` conversions, `NULLIF/TRIM()` for null handling, `UPPER(TRIM())` for result standardization, and conditional CASE expressions for overall_result normalization. Valid tables filtered clean records on multi-condition WHERE clauses — for QC results, requiring `overall_result IN ('PASS', 'FAIL')`, `potency_pct BETWEEN 80 AND 120`, and either a passing result or a failing result with a non-null `failure_reason`. Invalid tables captured the complement set. Row counts were verified: from 200 total QC records, 183 were valid and 17 were invalid. Valid batch tables were then written to S3 as Parquet using Hive `STORED AS PARQUET` with explicit `LOCATION` clauses, confirming the final batch layer export listing under `s3://pharma-data-lake-kritika/hive_exports`.

Phase 4 — Athena Analytics and QuickSight Dashboards: Amazon Athena was connected to the Glue Data Catalog targeting the `pharma_db_kritika` database. External tables were created in Athena for the streaming and batch Parquet datasets using `ROW FORMAT SERDE` with appropriate SerDe properties and `S3 LOCATION` clauses. Six analytical queries were developed and executed: (1) high-value batches with temperature limit breaches in the last hour nearing expiry; (2) distributors projected to run out of stock based on sales velocity and current inventory; (3) recall sales impact comparison (affected period vs. normal baseline); (4) distributors with repeated temperature violations across multiple batches; (5) pharmacies purchasing unusually high quantities relative to historical patterns; and (6) active batch compliance posture across both QC and temperature requirements. QuickSight was connected to Athena via a new data source with `primary` workgroup configuration. Two SPICE-imported datasets were created: `v_cold_chain_compliance` and `v_regional_sales_p`, both ingested at 100% success with 8,400 rows. Two analyses were published: **`v_cold_chain_compliance_analysis`** displaying a batch-product breach table and an `actual_temp` vs. `event_time` trend chart, and **`v_regional_sales_performance_analysis`** showing quantity sold by product name as a vertical bar chart.

Table 2.9: Weekly Work Summary — Pharmaceutical Supply Chain Streaming Platform

Week	Activity	Outcome
1	Kinesis Provisioning & EC2 Producer Development	Stream created; producer publishing temperature, sales, and inventory events at 30-second intervals; streaming confirmed via terminal output
2	EMR Spark Consumer — Streaming Ingestion & Validation	Three typed streams cleaned, validated, and written to Hive Parquet tables with S3 checkpoints; data confirmed via Spark SQL queries
3	Sqoop Extraction & Hive Batch Pipeline	QC results and product master extracted from RDS to HDFS; four-tier Hive table hierarchy (raw, clean, valid, invalid) built for all batch datasets
4	S3 Export, Athena Analytics & QuickSight Dashboards	Valid batch tables exported to S3 Parquet; six Athena analytical queries developed; two QuickSight SPICE analyses published

3.6 Solution Provided

The platform delivers four tightly integrated technical capabilities that collectively address all identified pharmaceutical supply chain challenges.

Real-Time Cold Chain Monitoring via Kinesis + Spark Structured Streaming: Temperature readings from IoT sensors arrive on the Kinesis stream every 30 seconds and are processed in 120-second micro-batches by the Spark consumer. Within each micro-batch, the `temperature_c` column is computed by conditionally converting Fahrenheit values to Celsius using the formula $(\text{temperature} - 32) * 5/9$ and applying a physical validity filter. An `alert_flag` boolean is set by Spark when the normalized Celsius value falls outside the batch-specific `min_temp_c` / `max_temp_c` thresholds defined in the batch master. This delivers sub-3-minute cold chain breach detection, compared to the hours-long delay in manual audit-based systems, directly supporting continuous regulatory compliance evidence generation.

Unified Batch-Centric Data Spine via `batch_id` Joins: The `batch_id` field is the common natural key across all seven datasets in the platform. The Athena analytical layer joins `temp_valid` to `batch_valid` on `batch_id` to correlate temperature readings against batch-specific cold chain specifications. `batch_valid` joins to `product_valid` on `batch_id` to enrich compliance records with product-level metadata. `pharma_valid` (sales) joins to `batch_valid` to link transactions to batch QC status. `inventory_daily_valid` joins to `batch` records to compute stock coverage by batch. This unified linkage enables true end-to-end product lifecycle traceability — from raw material batch to patient dispensing — within a single analytical query, a capability absent from all predecessor systems.

Four-Tier Hive Table Architecture for Governed Batch Processing: Each batch dataset passes through four Hive table tiers. The `_raw` external table preserves the original textfile data from HDFS with no modifications, serving as a tamper-proof source of record. The `_clean` table applies type casting, string trimming (`TRIM()`), null replacement (`NULLIF()`), and result standardization (`UPPER(TRIM())` for QC results) using Hive CTAS. The `_valid` table applies multi-condition business rules — for QC results, this includes potency range validation (`potency_pct BETWEEN 80 AND 120`), result validity (`overall_result IN ('PASS', 'FAIL')`), and failure reason presence for failed results — retaining only analytically trustworthy records. The `_invalid` table captures the exact complement set for governance investigation. From 200 QC records, 183 were validated (91.5% pass rate) and 17 were quarantined, providing precise data quality quantification.

Automated Recall Impact Analysis via Cross-Domain Joins: Athena Query 3 — “How did a recall affect sales in the last 2 hours compared to normal?” — joins `pharma_valid` (sales streaming data) against `batch_valid` filtered on `batch_status = 'RECALLED'`, computes revenue during the recall window, and compares it against historical baseline revenue for the same products and pharmacies. This query executes in seconds rather than the 24–72 hours required by manual recall impact processes, reducing both patient safety risk and revenue uncertainty. The result set directly feeds the recall impact list required for regulatory notification.

As shown in Figure ??, the Spark consumer correctly classifies and routes streaming records into typed Hive tables, while the QuickSight cold chain compliance dashboard displays batch-level temperature breach records and event-time temperature trend charts for operational monitoring.

3.7 Results

The platform was validated end-to-end across both the streaming and batch processing lanes. The following outcomes were confirmed through systematic testing and query-level verification.

The Kinesis producer successfully emitted interleaved temperature, sales, and inventory records, confirmed by sequential terminal output. The Spark structured streaming consumer correctly split the unified stream into three typed DataFrames, applied stream-specific validation, and populated all three Hive tables (`temperature_streaming`, `sales_streaming`, `inventory_streaming`) in `pharma_streaming_db` with correctly typed and partitioned records. A `SELECT * FROM inventory_streaming` query in `spark-sql` returned 6 rows with correct `RECEIPT`, `DISPATCH`, and `RETURN` change types, valid stock deltas, and proper `partition_date` values. The S3 bucket

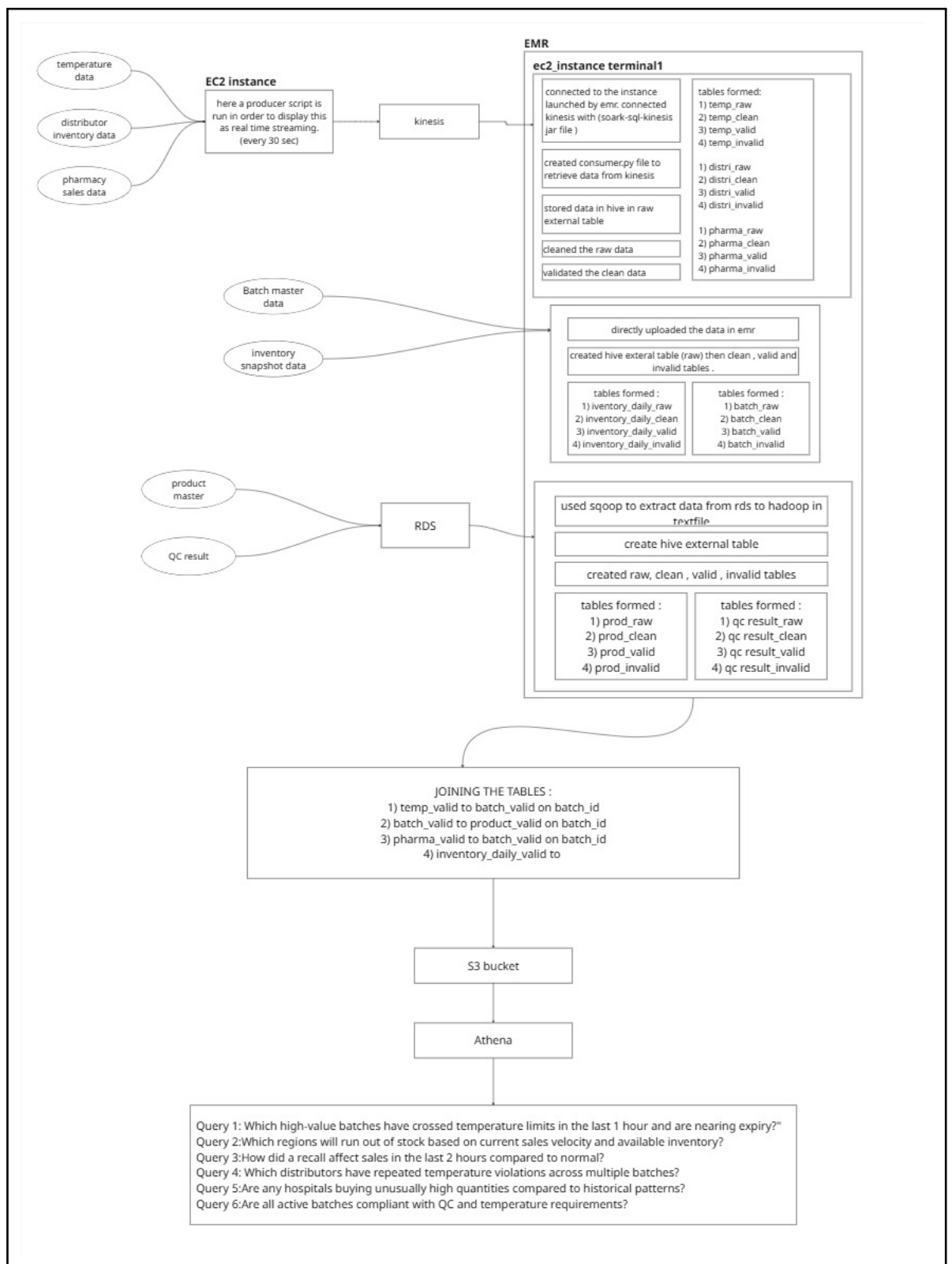


Figure 2.6: Detailed diagram of architecture

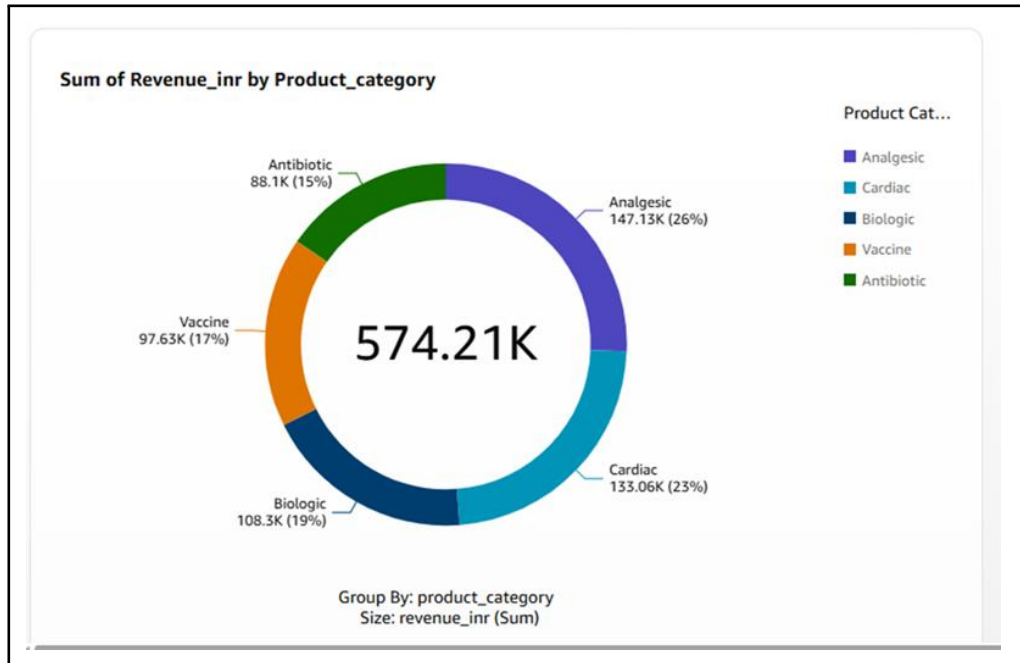


Figure 2.7: QuickSight — Cold Chain Compliance and Regional Sales Analysis

confirmed successful micro-batch writes under `streaming_kinesis/inventory/`, `streaming_kinesis/sales/`, and `streaming_kinesis/temperature/` with Snappy-compressed Parquet files.

On the batch side, Sqoop successfully extracted both the `qc_results` and `product_master` tables from RDS to HDFS as textfiles. The four-tier Hive pipeline was validated: `SELECT COUNT(*) FROM qc_results_clean` returned 200 rows; `SELECT COUNT(*) FROM qc_results_valid` returned 183; and `SELECT COUNT(*) FROM qc_results_invalid` returned 17, confirming a 91.5% valid rate. The `product_master_valid` table correctly filtered records on mandatory field non-nullability, unit price positivity, reorder point non-negativity, and cold chain temperature range logical consistency. The S3 Hive export listing confirmed all 12 table variants (4 tables × 3 tiers: raw, clean, valid, invalid) were present under `s3://pharma-data-lake-kritika/hive-exports/pharma.db.kritika`. Athena external table creation and querying succeeded for all datasets. QuickSight SPICE ingestion completed at 100% success for both datasets (8,400 rows imported, 0 rows skipped), and both analysis dashboards rendered correctly.

3.8 Conclusion

The Pharmaceutical Supply Chain Streaming Analytics Platform project successfully delivered a production-grade, dual-lane data engineering system that transforms pharmaceutical operations from reactive, batch-refreshed reporting to near-real-time decision intelligence. By integrating Amazon Kinesis Data Streams, Spark Structured Streaming,

Table 2.10: Pipeline Validation Test Cases — Pharmaceutical Supply Chain Platform

Test	Description	Expected Outcome	Status
TC-01	Kinesis producer record emission	Records published; all 3 source types confirmed	Pass
TC-02	Fahrenheit to Celsius normalization in stream	Correct <code>temperature_c</code> computed	Pass
TC-03	Temperature physical range filter (−50 to 100C)	Out-of-range records dropped	Pass
TC-04	Sales stream quantity/price positivity filter	Non-positive records excluded	Pass
TC-05	Inventory <code>change_type</code> whitelist filter	Only RECEIPT/DISPATCH/RETURN/ADJUSTMENT retained	Pass
TC-06	Hive Parquet micro-batch write with S3 checkpoint	Records persisted; checkpoint updated every 120s	Pass
TC-07	Sqoop RDS-to-HDFS extraction (QC + product)	Textfile records correctly written to HDFS	Pass
TC-08	QC results four-tier Hive pipeline	200 raw; 183 valid; 17 invalid	Pass
TC-09	Product master cold chain temperature validation	Records with <code>min_temp_c</code> > <code>max_temp_c</code> rejected	Pass
TC-10	S3 Parquet export of valid batch tables	All 12 table variants confirmed in S3 listing	Pass
TC-11	Athena cross-domain join on <code>batch_id</code>	Streaming and batch data correctly joined	Pass
TC-12	QuickSight SPICE ingestion (both datasets)	8,400 rows; 100% success; 0 rows skipped	Pass

Amazon EMR with Hive, Apache Sqoop, Amazon Athena, and Amazon QuickSight into a coherent, `batch_id`-linked analytical platform, the system addresses all four identified pharmaceutical supply chain challenges — cold chain monitoring, batch traceability, inventory visibility, and recall impact assessment — within a single, unified data architecture.

From an engineering standpoint, this project was the most technically advanced of the three internship engagements, requiring proficiency in real-time event streaming, Kinesis producer and consumer design, Spark Structured Streaming with Hive integration, JDBC-based batch extraction via Sqoop, multi-tier Hive ETL with complex business rule validation, cross-domain analytical query design in Athena, and cloud-native BI delivery via QuickSight. The combination of streaming and batch processing within a single cohesive platform reflects the Lambda Architecture pattern adopted by enterprise-grade pharmaceutical data platforms.

From a business and compliance standpoint, the platform delivers measurable impact

across four dimensions. Cold chain breach detection latency is reduced from several hours to under three minutes, enabling regulatory-grade continuous compliance evidence. End-to-end batch traceability is established through a single `batch_id` linkage across seven datasets. Recall impact assessment time is reduced from 24–72 hours to seconds via Athena cross-domain joins. Proactive inventory replenishment decisions become possible through real-time days-of-stock coverage computation. Together, these capabilities represent a significant reduction in compliance risk, patient safety exposure, and financial uncertainty for pharmaceutical supply chain operations.

BIBLIOGRAPHY

A. PLAGIARISM REPORT

Breast_Cancer_SL.docx

ORIGINALITY REPORT

9%

SIMILARITY INDEX

3%

INTERNET SOURCES

7%

PUBLICATIONS

3%

STUDENT PAPERS

PRIMARY SOURCES

1

Submitted to Tulane University

Student Paper

1%

2

Sumit Garethiya, Himanshu Agrawal, Shilpa Gite, Suresh. V, Amit Kudale, Gaurav Wable, Girishchandra R. Yendargaye. "Affordable system for alerting, monitoring and controlling heat stroke inside vehicles", 2015 International Conference on Industrial Instrumentation and Control (ICIC), 2015

Publication

1%

3

Abdul, Abdullah. "Using Machine Learning Algorithms to Find Novel Biomarkers for Breast Cancer Using RNA-Seq Dataset", Morgan State University, 2023

Publication

1%

4

H L Gururaj, Francesco Flammini, V Ravi Kumar, N S Prema. "Recent Trends in Healthcare Innovation", CRC Press, 2025

Publication

1%

5

Mahmoud Darwich, Magdy Bayoumi. "An evaluation of the effectiveness of machine learning prediction models in assessing breast cancer risk", Informatics in Medicine Unlocked, 2024

Publication

1%

6

doctorpenguin.com

Internet Source

1%

7

scienceofbiogenetics.com

Internet Source

1%

Submitted to Nottingham Trent University

A Machine Learning-Based Predictive Model for Early Detection of Breast Cancer

1st Aaryan Tamhane
Computer Science
Symbiosis Institute of Technology
Nagpur India
aaryan.tamhane29@gmail.com

2nd Advait Kulkarni
Computer Science
Symbiosis Institute of Technology
Nagpur India
kulkarniadvait93@gmail.com

3rd Soham Shrawankar
Computer Science
Symbiosis Institute of Technology
Nagpur India
sohamshrawankar@gmail.com

4th Sargkumar Badhiye
Computer Science
Symbiosis Institute of Technology
Nagpur India
sargkumar.badhiye@sitnagpur.siu.edu.in

5th Pradnya Borkar
Computer Science
Symbiosis Institute of Technology
Nagpur India
pradnyaborkar2@gmail.com

Abstract—Breast cancer is one of the biggest health risks today, and early diagnosis is essential for a better prognosis of the disease. Data-driven approaches such as machine learning emerge successful in diagnosis and breast cancer prediction. This section assesses several general classifiers for prediction such as Random forest (RF), Logistic Regressions (LR) etc. algorithms explored on breast cancer datasets. The dataset is a fine needle aspiration (Biopsy) dataset namely Wisconsin Breast Cancer dataset, which was obtained from the UCI Machine Learning (ML) database. Discussions on the comparisons of different algorithms, feature selection methods, and performance measures are also covered.
Index Terms—Breast-cancer, Machine-learning, Classification algorithms, SVM, Random forest, Feature selection, Data mining.

I. INTRODUCTION

Breast-cancer is a serious and common type of cancer that is a great cause of death among women worldwide [1]. World Health Organization specifies it as a leading cause of death, responsible for around the fifteenth percent of all cancer death rates in women [5]. Timely and accurate detection followed by judicious handling of the disease path in the first stages, improves prognosis and survival rates- vastly [1]. While the accuracy, consistency, and efficiency of traditional diagnostic methods, such as mammography, are problematic in the early stages of the disease, non-invasive methods powered by modern technologies usually demonstrate the strengths of all the mentioned indicators [2]. Figure 1 shows the incidence of breast cancer cases in different countries like South America, North America, East Asia and South-Central Asia. Cancer is a broad term that refers to a group of diseases that share a common characteristic: the abnormal proliferation of body cells or cell multiplication so the body cannot regulate it properly. Unpacking that way, cancerous cells depart from the regulated growth and division orders that healthy cells follow and divide and multiply, basically, at an exorbitant rate, which ends up in the formation and enlargement of malignant tumors or neoplasms [14]. There are various types of cancers

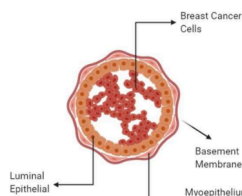


Fig. 1. Cancer Cell

that are named on the basis of the location of concurrence in the human body. For example, lung cancer, breast cancer, colorectal cancer, prostate cancer and skin cancer are the most common type of cancers worldwide. Figure 2 shows some common types of cancers and how common they are based on the number of cases worldwide per year. Cancer was a disease that led to the genome material which was started due to cell division and growth pathway interference. Alterations in genes were the cause of these mutations. These genetic modifications identify the cells for the new job and the specific task, they emphasize controlled cellular growth as well as its checks cells that are healthy contain a complex system of checks and balances which is supposed to govern orderly growth and division; however, in cancer cells, this is disturbed by genetic mutations and leads to uncontrolled growth. In the process, the cells as a whole turn into mutineers, breaching the jurisdiction of usual laws and proliferating endlessly, and eventually become tumors or spread to other

B. AI SIMILARITY REPORT

Breast_Cancer_SL.docx

ORIGINALITY REPORT

9%

SIMILARITY INDEX

3%

INTERNET SOURCES

7%

PUBLICATIONS

3%

STUDENT PAPERS

PRIMARY SOURCES

1

Submitted to Tulane University

Student Paper

1%

2

Sumit Garethiya, Himanshu Agrawal, Shilpa Gite, Suresh. V, Amit Kudale, Gaurav Wable, Girishchandra R. Yendargaye. "Affordable system for alerting, monitoring and controlling heat stroke inside vehicles", 2015 International Conference on Industrial Instrumentation and Control (ICIC), 2015

Publication

1%

3

Abdul, Abdullah. "Using Machine Learning Algorithms to Find Novel Biomarkers for Breast Cancer Using RNA-Seq Dataset", Morgan State University, 2023

Publication

1%

4

H L Gururaj, Francesco Flammini, V Ravi Kumar, N S Prema. "Recent Trends in Healthcare Innovation", CRC Press, 2025

Publication

1%

5

Mahmoud Darwich, Magdy Bayoumi. "An evaluation of the effectiveness of machine learning prediction models in assessing breast cancer risk", Informatics in Medicine Unlocked, 2024

Publication

1%

6

doctorpenguin.com

Internet Source

1%

7

scienceofbiogenetics.com

Internet Source

1%

Submitted to Nottingham Trent University

A Machine Learning-Based Predictive Model for Early Detection of Breast Cancer

1st Aaryan Tamhane
Computer Science
Symbiosis Institute of Technology
Nagpur India
aaryanamhane29@gmail.com

2nd Advait Kulkarni
Computer Science
Symbiosis Institute of Technology
Nagpur India
kulkarniadvait93@gmail.com

3rd Soham Shrawankar
Computer Science
Symbiosis Institute of Technology
Nagpur India
sohamshrawankar@gmail.com

4th Sargkumar Badhiye
Computer Science
Symbiosis Institute of Technology
Nagpur India
sargkumar.badhiye@sitnagpur.siu.edu.in

5th Pradnya Borkar
Computer Science
Symbiosis Institute of Technology
Nagpur India
pradnyaborkar2@gmail.com

Abstract—Breast cancer is one of the biggest health risks today, and early diagnosis is essential for a better prognosis of the disease. Data-driven approaches such as machine learning emerge successful in diagnosis and breast cancer prediction. This section assesses several general classifiers for prediction such as Random forest (RF), Logistic Regressions (LR) etc. algorithms explored on breast cancer datasets. The dataset is a fine needle aspiration (Biopsy) dataset namely Wisconsin Breast Cancer dataset, which was obtained from the UCI Machine Learning (ML) database. Discussions on the comparisons of different algorithms, feature selection methods, and performance measures are also covered.
Index Terms—Breast-cancer, Machine-learning, Classification algorithms, SVM, Random forest, Feature selection, Data mining.

I. INTRODUCTION

Breast-cancer is a serious and common type of cancer that is a great cause of death among women worldwide [1]. World Health Organization specifies it as a leading cause of death, responsible for around the fifteenth percent of all cancer death rates in women [5]. Timely and accurate detection followed by judicious handling of the disease path in the first stages, improves prognosis and survival rates- vastly [1]. While the accuracy, consistency, and efficiency of traditional diagnostic methods, such as mammography, are problematic in the early stages of the disease, non-invasive methods powered by modern technologies usually demonstrate the strengths of all the mentioned indicators [2]. Figure 1 shows the incidence of breast cancer cases in different countries like South America, North America, East Asia and South-Central Asia. Cancer is a broad term that refers to a group of diseases that share a common characteristic: the abnormal proliferation of body cells or cell multiplication so the body cannot regulate it properly. Unpacking that way, cancerous cells depart from the regulated growth and division orders that healthy cells follow and divide and multiply, basically, at an exorbitant rate, which ends up in the formation and enlargement of malignant tumors or neoplasms [14]. There are various types of cancers

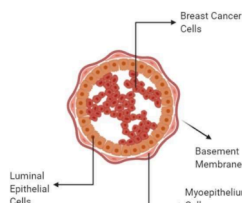


Fig. 1. Cancer Cell

that are named on the basis of the location of concurrence in the human body. For example, lung cancer, breast cancer, colorectal cancer, prostate cancer and skin cancer are the most common type of cancers worldwide. Figure 2 shows some common types of cancers and how common they are based on the number of cases worldwide per year. Cancer was a disease that led to the genome material which was started due to cell division and growth pathway interference. Alterations in genes were the cause of these mutations. These genetic modifications identify the cells for the new job and the specific task, they emphasize controlled cellular growth as well as its checks cells that are healthy contain a complex system of checks and balances which is supposed to govern orderly growth and division; however, in cancer cells, this is disturbed by genetic mutations and leads to uncontrolled growth. In the process, the cells as a whole turn into mutineers, breaching the jurisdiction of usual laws and proliferating endlessly, and eventually become tumors or spread to other

C. PROJECT SCREENSHOTS

